Retriva

Design session #1: Initial design

Document version: 0.9.4 (20260331)

Andrea Marson

am.dev.75@gmail.com

Session item	**Value**	**Notes**
Start date	20260314	
chatbot	Microsoft Copilot	
model	GPT 5.4 Think	

Table of Contents

1 Starting requirements

You said:

You are an computer scientist who is expert of AI technologies. My final goal is to implement a RAG-featured chatbot to be used in my company. Requirements: - It must use open-weight models for chatting with users and for storing embeddings. I refer to the fist model as "conversational model" and to the second as "embedding model". - The custom knowledge base consists of a website, implemented with Mediawiki, containing text and images, The website provide detailed, technical information about electronic boards and embedded systems. During the creation of embeddings, images must be analyzed thoroughly to extract detailed information because they can provide important technical data. For instance, images can represent Cartesian charts showing measurements of electrical parameters of such boards. Attached you can find an example. - It must be fully deployed locally for confidential matters - It must retrieve information to answer users' questions from the knowledge base only. - User must be able to chat in English or Italian - The chat application with which users interact must be implemented as a web application so that users access it by using their browsers. To implement it, a simple-to-use framework such as Streamlit must be used. - The system must be deployed on a private local network and must be accessible by the local users only - If it is necessary to code parts of the system, they must be written in Python - The models should be accessed via OpenAI API so that if I replace them the software interface does not change. Before implementing the production-grade system, I want to implement an experimental version that I also refer to as proof of concept (PoC). The PoC must allow me to try different approaches flexibly. The experimental version — entirely or partially — can be deployed on the cloud because no reserved information are used for testing. The PoC must makes use of a knowledge based built upon a locally-available mirror of the Mediawiki-based website wiki.dave.eu. This website if pretty similar to the one that contains confidential information and that will be used for the production-grade system. I would like to start with a relatively simple implementation for the first version of the PoC and to increase its complexity gradually by adding advanced RAG techniques such as hybrid search combining the strengths of vector-based semantic search with traditional keyword-based methods.

Copilot said:

Copilot

Absolutely — this is a **very good fit for a phased RAG program**, and your constraints are coherent.

What you want is essentially a **two-stage roadmap**:

1. **PoC / experimental stack**: easy to iterate, partly cloud-acceptable, flexible enough to compare chunking/retrieval/image strategies.
2. **Production stack**: same architecture patterns, but fully local, private-network-only, no data leaving premises.

Below I propose a design that starts simple and grows in capability without forcing a rewrite.

- **1) Executive recommendation**
- **Recommended overall direction**

For the **first PoC version**, I strongly recommend this baseline:

- **Conversational model**: one **open-weight instruction-tuned model** served behind an **OpenAI-compatible API**.
- **Embedding model**: **BGE-M3** (multilingual, long-context, and capable of dense + sparse retrieval features). BGE-M3 supports **100+ languages**, **up to 8192 tokens**, and can produce **dense**, **sparse**, and **multi-vector** retrieval representations.huggingface+1
- **Model serving**: **vLLM**, because it exposes **OpenAI-compatible endpoints** including **Chat Completions** and **Embeddings**, which matches your requirement that the software interface should remain stable if you swap models later.vllm+1
- **Vector store / retrieval engine**: **Qdrant**, because it supports **hybrid search** by combining dense vectors with sparse/BM25-style search and supports fusion methods such as **RRF**.qdrant+1
- **Web app**: **Streamlit** for the first UI, because it is fast to iterate with and perfectly adequate for internal users in a PoC.
- **Knowledge ingestion**: parse the **local MediaWiki mirror** using the **MediaWiki API** (revisions + parse/images), preserving page/section/image metadata. MediaWiki's API exposes revision content and parsed page outputs, including page images.mediawiki+1

This stack satisfies your core constraints while keeping the first version simple.

- **2) Important design principle: treat images as first-class knowledge**

This is the most important part of your requirement.

The attached example image is not "decorative"; it carries **technical evidence**. Objectively, the image shows:

- a **line chart**
- x-axis labeled **"Time [s]"**, ranging roughly from **0 to 180**
- y-axis labeled **"SoM Power [W]"**, ranging roughly from **0.0 to 3.0**
- a **blue trace** staying mostly around ~**1.65–1.72 W**
- several short spikes, with the largest visible peaks roughly near ~**2.2 W**
- notable bursts around approximately **100–120 s** and again close to **175–180 s**

That means your ingestion pipeline cannot just "attach the image filename" — it must convert images into **retrievable textual knowledge** and, where possible, **structured data**.

- **Therefore, for images in the PoC, do this:**
- **V1 image strategy (simple but effective)**

For each image:

1. **OCR any visible text**
    - axis labels
    - legends
    - annotations
    - table labels
    - captions embedded inside the image
2. **Run a local open-weight VLM** to produce:
    - a **detailed caption**

- a **technical interpretation**
- a **structured summary**
- if the image is a chart, extract:
  - chart type
  - axes and units
  - approximate ranges
  - qualitative behavior (steady / rising / oscillatory / spikes)
  - any clearly visible peaks or anomalies

3. Store the result as **text chunks** linked to:
   - page title
   - section title
   - image filename
   - image position on page
   - image caption (if available)
   - source page URL / revision ID

This is the best trade-off for a first version.

- **Suggested local image-understanding model**

A strong candidate is **Qwen2.5-VL**, which is an open vision-language model family. The Hugging Face documentation describes Qwen2.5-VL as a multimodal model family for image/video understanding, and external model descriptions note multilingual support for text inside images and strong capability on charts/documents.huggingface+1

For your use case, I would **not** use image embeddings in V1.
Instead, I would:

- turn images into **high-quality technical text**
- embed that text with the same **embedding model**

- retrieve it like normal text

This is simpler, easier to debug, and works surprisingly well.

- **3) PoC architecture (Version 1)**

Here is the **recommended first PoC architecture**.

- **Components**
- **A. Ingestion pipeline (Python)**

Reads the local mirror of the MediaWiki-based site and creates chunks.

Responsibilities

- read page titles and revisions
- extract page content and sections
- extract linked images
- fetch image metadata/captions
- run OCR + VLM description on images
- create chunk records
- create embeddings
- upload to Qdrant

MediaWiki extraction note:
MediaWiki exposes revision content through prop=revisions, and parsed page output — including images — via action=parse.mediawiki+1

- **B. Retrieval index**

Use **Qdrant** collections with:

- **dense vector** field
- **sparse vector** field (optional in V1, enabled in V2)
- payload metadata:

- page_title
- section_title
- revision_id
- language
- source_url
- chunk_type (text, image_summary, table_summary, chart_summary)
- image_filename
- board_family
- product_name
- tags

Qdrant supports combining dense and sparse retrieval and fusing results using approaches such as **Reciprocal Rank Fusion**.qdrant+1

- **C. Chat backend**

Python service that:

1. receives user question from Streamlit
2. retrieves top chunks from Qdrant
3. builds a grounded prompt
4. calls the conversational model via **OpenAI-compatible API**
5. returns answer with **source citations**

- **D. Model serving**

Use **vLLM**.

vLLM provides an **OpenAI-compatible HTTP server** and supports:

- /v1/chat/completions

- /v1/embeddings
- and use with the **official OpenAI Python client** by changing base_url.vllm+1

This is exactly aligned with your "replace model without changing software interface" requirement.

- **E. Streamlit web UI**

The UI can initially include:

- chat panel
- answer with citations
- expandable retrieved chunks
- source page links
- language selector (EN / IT)
- debug mode for developers:
    - retrieved chunks
    - scores
    - image summaries included in context
    - latency

This "debug mode" will be incredibly useful during the PoC.

- **4) Recommended phased roadmap**

- **Phase 0 — Prepare the mirror and schema**

Goal: make ingestion deterministic.

- **Deliverables**
- local mirror of wiki.dave.eu

- page export pipeline
- image extraction pipeline
- chunk schema
- evaluation set of ~50–100 real technical questions
- **Why this matters**

The DAVE wiki is clearly a technical documentation resource focused on SOMs, SBCs, embedded systems, and related developer documentation.
That makes it an excellent proxy corpus for your confidential wiki.dave+1

- **Phase 1 — Minimal PoC (recommended first implementation)**

Goal: get an end-to-end working RAG chatbot quickly.

- **Features**
- local or cloud test deployment
- MediaWiki text extraction
- section-aware chunking
- image captioning + OCR -> textual summaries
- embeddings with **BGE-M3**
- **dense vector retrieval only**
- answer generation with one conversational open-weight model
- Streamlit UI
- citations back to source wiki pages
- "I don't know based on the available documentation" fallback
- **Why BGE-M3 is a strong fit**

BGE-M3 is especially relevant here because it is:

- multilingual (good for **English + Italian**),
- long-context for chunking flexibility,

- and later can support hybrid retrieval strategies from the same embedding family.huggingface+1
- **What I would keep simple in V1**
- no reranker yet
- no hybrid search yet
- no agent/tool calling
- no multimodal retrieval at query time
- no query rewriting initially

This lets you validate the corpus processing first.

- **Phase 2 — Better retrieval quality**

Goal: improve recall and precision.

- **Add:**

1. **Hybrid search**
    - dense semantic retrieval + sparse/BM25 retrieval
    - fusion with RRF

Qdrant explicitly supports dense+sparse hybrid retrieval and RRF-style fusion.qdrant+1

2. **Reranking**
    - use an open reranker after first-stage retrieval
    - rerank top 20–50 chunks down to top 5–8 for prompt context
3. **Metadata filters**
    - by product family
    - by board name
    - by software/hardware section
    - by language

    - by document type

- **Phase 3 — Better multimodal knowledge extraction**

Goal: improve understanding of technical images.

- **Add:**
- specialized prompt templates for:
    - charts
    - PCB photos
    - pinout diagrams
    - architecture block diagrams
    - timing diagrams
    - screenshots of settings/tools
- chart-specific extraction output in JSON
- table extraction from images
- relation linking:
    - image ↔ nearby paragraph
    - image ↔ caption
    - image ↔ section title
- **Example of chart summary JSON**

For a chart like the one you attached, a record might look like this:

JSON

Then embed:

- the raw JSON text
- a natural-language summary
- the image caption

- the surrounding section text

That makes the chart searchable by many styles of question.

- **Phase 4 — Production-grade local deployment**

Goal: move from PoC to confidential internal system.

- **Production changes**
- all services deployed **on-premises**
- private LAN only
- no internet egress from inference / index containers
- local authentication / SSO if needed
- encrypted storage for vector DB and logs
- document refresh pipeline from internal MediaWiki
- audit logging
- backup / restore
- monitoring
- GPU sizing and failover
- **Likely deployment topology**
- streamlit-ui
- rag-api
- ingestion-worker
- vllm-chat
- vllm-embed
- vlm-image-worker
- qdrant
- reverse-proxy

- optional auth-proxy

This should be containerized from day one so the PoC transitions cleanly.

- **5) Key architectural choices I recommend**
- **Choice A — Keep chat and embedding models decoupled**

Use **separate model services** for:

- conversational generation
- embeddings
- optionally image understanding

Why:

- easier benchmarking
- easier replacement
- clearer resource allocation
- better observability

Even if both are accessed through OpenAI-style APIs, treat them as separate backend roles.

- **Choice B — Ground answers strictly in retrieved context**

You wrote: **"It must retrieve information to answer users' questions from the knowledge base only."**

This is the correct target, but one nuance matters:

An LLM cannot be forced to forget pretraining knowledge in an absolute mathematical sense.

So in practice, you should enforce this at the **application level**:

- **Grounding policy**
- answer **only from retrieved passages**

- require **citations**
- if evidence is insufficient, answer:
    - "I cannot answer from the available knowledge base."
- optionally expose the retrieved excerpts to the user

This is the best practical way to meet your requirement.

- **Choice C — Use bilingual retrieval, not bilingual translation pipelines**

Because the users can ask in **English or Italian**, the embedding layer should support both languages natively.

That is exactly why **BGE-M3** is attractive: it is multilingual and built for retrieval workloads.huggingface+1

For V1:

- store chunks in original language
- preserve page language metadata
- retrieve across both languages using multilingual embeddings

Later you can add:

- query expansion in the other language
- bilingual answer generation
- language-aware reranking

- **6) Concrete recommendation for model selection**

Because you want flexibility, I would not hard-code one "forever" model now.

- **My recommendation:**
- **Conversational model**

Benchmark **2–3 open-weight instruction models** that are reasonably multilingual and can be self-hosted.

Examples of families worth testing:

- Qwen instruct family
- Llama instruct family
- Mistral / Mixtral instruct family

For PoC, choose one model that gives:

- good English technical QA
- acceptable Italian
- manageable VRAM footprint
- stable instruction following
- **Embedding model**

Use **BGE-M3** first.
It is particularly aligned with your roadmap because it supports:

- multilingual retrieval
- long documents
- dense + sparse style capabilities for future hybrid retrieval.huggingface+1
- **Image understanding model**

Use an **open VLM** such as **Qwen2.5-VL** for image-to-text enrichment. The model family is documented for multimodal understanding, and descriptions emphasize strong capability on charts, layouts, and multilingual text in images.huggingface+1

- **7) Recommended chunking strategy**

For this domain, chunking quality matters a lot.

- **Start with:**
- **section-aware chunking**

- chunk size: roughly **400–800 tokens**
- overlap: **10–15%**
- preserve:
    - page title
    - section hierarchy
    - bullet lists
    - tables as text
    - code blocks as text
    - image summaries as adjacent chunks
- **Very important**

Do **not** mix unrelated sections into one chunk.

Technical wiki pages often contain:

- hardware features
- pin mappings
- BSP instructions
- power measurements
- software versions
- bootloader settings

If chunking is too coarse, retrieval quality drops sharply.

- **8) Evaluation plan (do this early)**

Before optimizing architecture, define an evaluation set.

- **Build a gold dataset of questions like:**
- "What is the idle power consumption of board X under condition Y?"
- "Which section describes UART pin configuration for module Z?"

- “What is the expected voltage/current range shown in the measurement chart?”
- “How do I boot Linux on board A using storage B?”
- “Which wiki page contains the thermal limitations of product C?”
- **Measure:**
- **retrieval hit rate** (does top-k contain the relevant chunk?)
- **grounded answer correctness**
- **citation correctness**
- **Italian query performance**
- **image-derived answer quality**

Without this dataset, it is hard to know whether hybrid search or better chunking is actually helping.

- **9) What I would implement first (my practical recommendation)**

If I were building this for your company, I would implement the PoC in this exact order:

- **PoC v1**
- Python ingestion pipeline for MediaWiki mirror
- text + image-summary chunk generation
- BGE-M3 embeddings
- Qdrant dense retrieval
- one open-weight chat model via vLLM
- Streamlit UI with citations
- explicit “insufficient evidence” fallback
- **PoC v2**
- add hybrid search in Qdrant

- add reranking
- add debug analytics dashboard
- **PoC v3**
- improve chart/table extraction
- add structured image metadata
- add evaluation automation
- **Production**
- move all services on-prem
- lock to private LAN
- add auth, monitoring, backups, refresh jobs

That path keeps the complexity under control.

- **10) A good reference architecture for your case**

Plain Text

vLLM's OpenAI-compatible server is especially valuable here because it allows your Python application to call local models with the same client pattern used for OpenAI-style APIs.vllm+1

- **11) Bottom line**
- **Best first PoC stack**
- **UI**: Streamlit
- **Backend**: Python
- **Model serving**: vLLM (OpenAI-compatible)
- **Embeddings**: BGE-M3
- **Retrieval DB**: Qdrant
- **Image understanding**: open VLM + OCR

- **Corpus source**: local mirror of wiki.dave.eu
- **Best first retrieval strategy**
- **dense retrieval only** for V1
  Then:
- **hybrid dense + sparse** in V2
- **reranking** in V2/V3
- **Best first multimodal strategy**
- **convert images into rich text + structured summaries**
- embed those summaries
- retrieve them as normal chunks

That is the simplest architecture that still respects your real requirement about technical images.

If you want, my next step can be one of these:

1. **design the full PoC architecture in more detail**,
2. **generate a Python project structure**,
3. **write a first docker-compose.yml for the PoC**, or
4. **draft the ingestion pipeline for MediaWiki + image analysis**.

If you want the most practical next step, I'd suggest:
"Please design the PoC architecture and folder structure in detail."

Sources

2 Considering Weaviate for vector DB

You said:

What do you think about using Weaviate as vector DB?

Copilot said:

Copilot

Yes — **Weaviate is a credible choice** for your project. In fact, for your roadmap it is **a good fit technically**, especially because it supports **self-hosting**, **hybrid search (vector + BM25)**, **structured filtering**, and it can work with **your own precomputed embeddings** instead of forcing you to use its built-in vectorizers.weaviate+3

That said, my honest opinion is:

Weaviate is a good option, but for your first PoC I would treat it as "strong alternative to Qdrant," not an obvious winner.weaviate+2

- **My short answer**
- **If you ask me "Would I allow Weaviate in this architecture?"**

Yes, absolutely. It can satisfy your main requirements: self-hosted deployment, local/private-network usage, hybrid retrieval, and compatibility with an architecture where models are served separately behind an OpenAI-style API.weaviate+2

- **If you ask me "Would I choose it for the first PoC?"**

Maybe, but only if you already know you want hybrid search and richer DB-side capabilities early. Otherwise, for a **very lean first PoC**, I still slightly prefer a simpler retrieval stack such as **Qdrant** because it keeps the experimentation surface smaller. This part is my architectural judgment rather than a documentation fact.

- **Why Weaviate fits your requirements**
- **1) Local / private deployment**

Weaviate can be run **locally with Docker** and also deployed on **Kubernetes** for longer-running or production-style deployments. The official docs explicitly describe local Docker setup and self-hosted deployment options.weaviate+1

For your production requirement — **private local network only** — that is absolutely compatible with Weaviate.weaviate+2

- **2) Hybrid search is built in**

Weaviate has **native hybrid search** that combines:

- **vector similarity search**
- **BM25 keyword search**
- score fusion in the same query flow.weaviate

It also exposes parameters such as:

- **alpha** to balance vector vs keyword weight
- fusion strategies such as **relativeScoreFusion** and **rankedFusion**.weaviate

That aligns very well with your plan to start simple and later add **advanced RAG techniques such as hybrid search**.weaviate

---

- **3) You can keep your own embedding model**

This is important for your constraints.

You said:

- the embedding model must be **open-weight**
- the software should not depend on changing APIs when models are replaced
- models should be accessed via **OpenAI API-style interfaces**

Weaviate does **not** force you to use its built-in vectorizers. The official quickstart explicitly allows importing **pre-computed vectors**, and the Docker Hub description also says you can upload **your own vectors**.weaviate+1

So your architecture could be:

- **vLLM** for /v1/embeddings with **BGE-M3**
- **vLLM** (or another OpenAI-compatible server) for chat completions
- **Weaviate** only as the retrieval/index layer.vllm+3

That is a clean separation of concerns.

- **4) It supports richer AI-database patterns**

Weaviate is more than "just a vector store." It stores **objects plus vectors**, supports structured filters, and exposes clients/REST APIs. The OpenAI cookbook page describes it as an open-source vector search engine storing both objects and vectors, and the Docker Hub page highlights built-in vector and hybrid search plus modular integrations.openai+1

That can be useful for your documentation domain, where metadata matters a lot:

- board family
- product name
- hardware/software section
- document type
- language
- page title
- revision id

Weaviate is well suited to combining semantic retrieval with metadata-based narrowing.openai+1

- **Where Weaviate is especially strong for your use case**
- **A. If you want hybrid retrieval early**

Because hybrid search is already a first-class feature in Weaviate, it can be attractive if you know in advance that you will soon move beyond plain dense retrieval.weaviate

For technical documentation, hybrid often helps because users ask things like:

- exact board names
- interface names like UART/SPI/I2C
- exact SoC references
- literal error strings
- version numbers
- part numbers

BM25 helps with those exact tokens, while vector search helps with semantic phrasing.weaviate

---

- **B. If you expect multimodal growth later**

Weaviate's messaging and ecosystem emphasize support for multiple data modalities and modular vectorization/inference integrations. The Weaviate site and Docker Hub materials mention support across text, images, audio, and video data modalities.weaviate+1

For your case, though, I would still recommend this design choice:

Do not rely on DB-native multimodal magic for V1.
Instead, use your image-analysis pipeline to convert images into **rich text + structured summaries**, then store those results in Weaviate like any other chunk.

That keeps your system explainable and easier to evaluate.

- **The caveats / what I would watch carefully**
- **1) Don't couple model inference to the database**

Weaviate offers modules and integrations with embedding/generative providers, and the local quickstart even shows running Weaviate with Ollama modules.weaviate+1

That is useful, but in **your** project I would **not** make Weaviate responsible for model inference.
Why?

Because your architectural requirement is very clear:

- open-weight models
- replaceable models
- consistent **OpenAI-style API**
- local deployment for production

So I would keep:

- **model serving outside the DB**
- Weaviate only for **storage + retrieval**.

That preserves clean boundaries and makes future replacement easier.

- **2) For a first PoC, Weaviate may be more platform-like than you need**

This is not a criticism — it is a trade-off.

Weaviate is powerful, but it can encourage a more feature-rich "AI database" setup. For a first PoC whose main purpose is **retrieval-quality experimentation**, you may prefer a narrower stack:

- ingest
- embed
- retrieve
- rerank
- answer

That is why I still see **Qdrant** as slightly simpler for a bare-bones first version, while Weaviate becomes more attractive if you already want richer hybrid/filtered/search-platform behavior.
The objective part is that **both** systems support hybrid retrieval patterns; Weaviate documents hybrid search with BM25+vector fusion, and Qdrant documents dense+sparse hybrid retrieval with fusion methods such as RRF.weaviate+2

- **3) Privacy details: check telemetry and configuration**

The local Weaviate quickstart notes that **telemetry is enabled by default** and refers users to learn more or opt out.weaviate

For your eventual confidential production environment, I would explicitly review:

- telemetry settings
- access control
- network exposure
- persistence path
- backup/restore
- logs containing prompts or retrieved content

This is true for any self-hosted component, but it is especially important in your case.

- **My recommendation specifically for your project**
- **Recommendation summary**
- **Use Weaviate if:**
- you want to adopt **hybrid search early**, not later.weaviate
- you want a richer **object + vector + filter** retrieval platform.openai+1
- you are comfortable treating the DB as a broader AI data platform rather than only a vector index.
- **Prefer Qdrant first if:**
- you want the **smallest possible PoC stack**
- you want retrieval experiments with minimal platform surface
- you want to keep the first version laser-focused on chunking, image textualization, and dense retrieval before adding hybrid search.

Qdrant also supports hybrid retrieval and RRF-style fusion, so you are not giving that future up.qdrant+1

- **What I would do if you choose Weaviate**

I would structure it like this:

1. **MediaWiki ingestion in Python**
    - page parsing
    - section-aware chunking
    - image OCR + VLM analysis
    - chart/table summaries
2. **Embedding generation outside Weaviate**
    - use **BGE-M3** via an OpenAI-compatible embeddings endpoint
    - store precomputed vectors in Weaviate.huggingface+2
3. **Weaviate retrieval**
    - V1: dense search + filters
    - V2: hybrid search with BM25 + vector, tune alpha/fusion.weaviate
4. **Chat generation outside Weaviate**
    - open-weight conversational model via OpenAI-compatible API
    - grounded prompt with citations
5. **Streamlit web UI**
    - answer
    - sources
    - debug retrieval mode

That would be a sound architecture.

- **Final opinion**
- **My bottom line**

Weaviate is a strong and perfectly reasonable choice for your use case.weaviate+2

If you told me:

- "I want a clean first PoC with minimal moving parts,"
  I would still lean **Qdrant**.

If you told me:

- "I already know hybrid search and richer filtering will matter quickly, and I want a more full-featured AI database from the beginning,"
  then **Weaviate becomes very compelling**.weaviate+2

If you want, I can do one of these next:

1. **compare Weaviate vs Qdrant specifically for your architecture**,
2. **propose a Weaviate-based PoC architecture**, or
3. **draft the schema/collections design for Weaviate for MediaWiki pages + image summaries**.

Sources

3 vLLM vs OLLaMA

You said:

I never heard of vLLM before. Is it better than OLLaMA for this project?

Copilot said:

Copilot

Short answer: **for *your* project, I would lean toward vLLM over Ollama**, especially once you move beyond a toy PoC and want a stable **OpenAI-compatible serving layer** for both the conversational model and the embedding model.vllm+3

That said, **Ollama is easier to start with**, and for a very quick demo it can absolutely work. Ollama's own docs position it as "the easiest way to get up and running" with local models, while vLLM positions itself as a fast serving engine with production-oriented features such as high-throughput serving, tensor parallelism, streaming, quantization, and an OpenAI-compatible server.ollama+1

- **My practical recommendation**
- **If your question is "Which one is better for this company RAG project?"**

vLLM is the better fit overall. It is a stronger match for these requirements:

- **OpenAI-style API as the main contract**,
- **separate chat and embeddings endpoints**,
- **future model replacement without changing application code**,
- **higher-performance local serving**,
- and a path from **PoC to production**.vllm+1
- **If your question is "Which one is easier to try tonight?"**

Ollama is easier. It ships with a very approachable local experience, its API is automatically served on localhost:11434, and its docs are optimized for quick startup.ollama+2

So the cleanest summary is:

- **Ollama = easier developer experience**
- **vLLM = stronger serving architecture for your target system**ollama+1

- **Why I lean toward vLLM for your project**
- **1) OpenAI compatibility is more central in vLLM**

Your requirement is very explicit:

"The models should be accessed via OpenAI API so that if I replace them the software interface does not change."

vLLM explicitly documents an **OpenAI-compatible server** supporting **Chat Completions**, **Responses**, and **Embeddings**, and shows usage with the official openai Python client by changing base_url.vllm+1

Ollama also offers OpenAI compatibility, **but its own documentation labels that compatibility as experimental and subject to major adjustments including breaking changes**.ollama+1

For your architecture, that distinction matters a lot. If your application code should stay stable while models change, I would prefer the platform whose OpenAI-compatible interface is more clearly a first-class serving story. Based on the official docs, that points to **vLLM**.vllm+3

- **2) vLLM is built more like an inference server; Ollama is built more like a local model runtime**

vLLM describes itself as a **fast and easy-to-use library for LLM inference and serving**, emphasizing **state-of-the-art serving throughput**, **PagedAttention**, **continuous batching**, **quantization**, **tensor parallelism**, and an **OpenAI-compatible API server**.readthedocs

Ollama, by contrast, describes itself as the easiest way to run models locally and provides a stable native API on localhost:11434/api, plus an experimental OpenAI-compatibility layer.ollama+2

That is why my mental model is:

- **Ollama** = excellent local app/runtime experience
- **vLLM** = inference-serving infrastructure layerreadthedocs+1

For an internal RAG system with multiple services, that infrastructure emphasis is a better match.

- **3) vLLM fits your "chat model + embedding model" separation very well**

Your design already distinguishes:

- a **conversational model**
- an **embedding model**

vLLM's OpenAI-compatible server explicitly supports **/v1/chat/completions** and **/v1/embeddings**.vllm+1

Ollama certainly supports embeddings too, and its native API has **/api/embed** plus docs dedicated to embeddings for semantic search, retrieval, and RAG.
However, Ollama's OpenAI compatibility layer is documented as experimental, whereas your requirement is specifically about **using OpenAI API as the long-term interface contract**.ollama+1ollama+1

So again, for your stated architecture, **vLLM lines up more cleanly**.vllm+1

- **4) vLLM looks more appropriate for production scaling**

vLLM advertises features such as:

- **high-throughput serving**,
- **continuous batching**,
- **tensor parallelism for distributed inference**,

- **streaming outputs**,
- **quantization options**,
- **prefix caching**,
- and **multi-LoRA support**.readthedocs

I want to be careful here: I am **not** claiming Ollama cannot be used in production. Ollama can absolutely run local models and serve APIs.
But from the documentation and positioning alone, **vLLM is much more clearly targeted at serving workloads where throughput, batching, and scaling behavior matter**.ollama+1readthedocs+1

For an internal chatbot used by multiple employees on a private network, that matters.

- **Where Ollama is genuinely attractive**
- **1) It is easier to install and use**

Ollama's docs are very straightforward: install Ollama, run a model, and the API is available on http://localhost:11434/api.ollama+2

For a PoC, this simplicity is very appealing. If you want to validate:

- model quality,
- prompt templates,
- basic RAG flow,
- image analysis with a VLM,

Ollama can help you get there quickly.ollama+1

- **2) Its native API includes embeddings and is RAG-friendly**

Ollama has official embeddings docs and an **/api/embed** endpoint. Its documentation explicitly says embeddings are for **semantic search, retrieval, and RAG**, and recommends models such as embeddinggemma, qwen3-embedding, and all-minilm.ollama+1

So if you were willing to use **Ollama's native API** rather than insisting on OpenAI compatibility as the core contract, Ollama would become much more attractive for a first PoC.ollama+2

- **3) Vision support is convenient for image-heavy experiments**

Ollama's OpenAI-compatibility docs include examples for **vision models** through chat completions with image inputs, and its capability pages cover **Vision**, **Structured Outputs**, **Tool calling**, and **Embeddings**.ollama+1

That can be convenient if you want to prototype the **image analysis stage** of your wiki-ingestion pipeline quickly.

- **The single most important difference for your case**

Here is the point I would base the decision on:

- **Your project wants OpenAI API compatibility as a long-term software abstraction**
- vLLM: **OpenAI-compatible server is central and explicit**, including chat and embeddings.vllm+1
- Ollama: **OpenAI compatibility is experimental**, while the native Ollama API is the more fundamental interface.ollama+2

That makes **vLLM the safer architectural choice** for your long-term design.

- **My recommended strategy for you**
- **Best option for your roadmap**

I would do this:

- **PoC**
- If you want **maximum speed and minimum friction**, you can start with **Ollama** for quick experiments.ollama+1

- If you want the PoC to already resemble the future production serving layer, start directly with **vLLM**.vllm+1
- **Production-oriented path**

For the final internal RAG system, I would favor:

- **vLLM for chat serving**
- **vLLM or another OpenAI-compatible embedding server for embeddings**
- **Qdrant or Weaviate for retrieval**
- **Streamlit for the first web UI**vllm+4

- **My honest verdict**
- **Is vLLM "better" than Ollama in general?**

Not universally. They solve slightly different primary problems.readthedocs+1

- **Is vLLM better for your specific project?**

Yes, probably. Because your project emphasizes:

- **OpenAI-compatible interfaces**,
- **replaceable backend models**,
- **RAG with both chat and embeddings**,
- **local deployment**,
- and eventual **production hardening**.vllm+3
- **Is Ollama still useful to you?**

Definitely yes. If you want the easiest path to quickly test models and multimodal prompts on a workstation, Ollama is still a great tool.ollama+2

- **My recommendation in one sentence**

Use Ollama if you want the fastest possible local prototype; use vLLM if you want the serving layer to match your final architecture.ollama+3

If you want, I can do either of these next:

1. **compare vLLM vs Ollama specifically for your PoC architecture**,
2. **propose a concrete stack using vLLM**, or
3. **give you a step-by-step migration path: start with Ollama, move to vLLM later**.

Sources

4 IBM Granite family

You said:

What do you think about using IBM Granite family of models?

Copilot said:

Copilot

Yes — **I think the IBM Granite family is a very serious option for your project**, and in one respect it is unusually attractive: IBM is not offering only a chat model, but a **coherent family** that includes **language models, embedding models, vision models, document-conversion models, and guardrail models**, all positioned for enterprise use and released under **Apache 2.0**. That breadth is unusually convenient for a company RAG roadmap like yours.ibm+4

My short verdict is this: **Granite is one of the best model families to benchmark for your use case**, but I would **not** choose the entire stack "blindly" without side-by-side tests against a few alternatives. In particular, I think **Granite Embedding** is especially interesting for you, **Granite Vision / Docling** are very relevant for your image-heavy wiki, and **Granite 4.0 language models** are promising for the chatbot itself if you value efficiency, local deployment, and enterprise-friendly licensing.ibm+4

- **What I like about Granite for your project**
- **1) The family is unusually aligned with enterprise RAG**

IBM explicitly positions Granite as **open, performant, and trusted AI models built for business**, and the Granite catalog includes models for **RAG**, **semantic search**, **visual document understanding**, **document conversion**, and **guardrails**. Granite 4.0 is described as suitable for **RAG**, agents, and edge/local deployments, while the broader Granite site explicitly lists embeddings, vision, Docling, and Guardian as separate model lines.ibm+1

That matters for your roadmap because your problem is not just "pick one LLM." Your problem is really a **system design problem**:

- chatbot generation,

- multilingual retrieval,
- image/chart understanding,
- document ingestion,
- and groundedness / hallucination control.
  Granite gives you a plausible option in every one of those layers.ibm+4

- **2) Granite 4.0 language models are a strong *candidate* for the conversational model**

Granite 4.0 language models are described by IBM as **lightweight open foundation models** with **multilingual capabilities**, **RAG support**, **tool usage**, and **structured JSON output**. IBM's Granite docs also describe the family as optimized for enterprise tasks like **RAG** and agents, with models ranging from very small sizes up to **Granite-4.0-H-Small**, which IBM explicitly calls a workhorse model for RAG and agentic workloads.github+1

For your bilingual requirement, an important point is that Granite 4.0 model cards list support for **English, German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, and Chinese**. So **Italian support is present**, which is directly relevant for your user-facing chat.huggingface+1

My opinion, though, is nuanced: **I would absolutely benchmark Granite 4.0 for the conversational model, but I would not assume it wins by default**. It looks very attractive if you care about **small/efficient local models** and enterprise-friendly deployment, but you should still compare it against strong alternatives on your actual evaluation set, especially for technical QA in Italian and English. Granite 4.0's strength on **instruction following**, **tool calling**, **JSON output**, and **RAG-oriented use cases** makes it highly relevant to test.ibm+2

- **3) Granite Embedding is one of the most interesting parts of the family for you**

For your project, the most immediately compelling Granite component may actually be the **embedding line**. IBM's Granite Embedding collection is explicitly designed

for **semantic search and retrieval-augmented generation**, and IBM says the collection includes both embedding models and a **reranker** to improve relevance after retrieval.ibm

The **Granite Embedding 278M Multilingual** model is especially relevant because it supports **Italian** along with 11 other languages and produces **768-dimensional embeddings** for retrieval/search use cases. IBM's model card states that the supported languages include **English, German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, and Chinese**.ibm+1

For your requirements, that means Granite Embedding is a **real candidate** for the embedding model — particularly because you need **English + Italian** and want something deployable locally with an enterprise-friendly license. IBM also documents that Granite Embedding models are available under **Apache 2.0** and offers them for local use, including via Ollama.ibm+2

That said, I would benchmark Granite Embedding directly against **BGE-M3**, because BGE-M3's main advantage is that it supports **100+ languages**, **up to 8192 tokens**, and can simultaneously support **dense**, **sparse**, and **multi-vector** retrieval styles — which is useful if you plan to evolve into hybrid retrieval without changing families. Granite Embedding looks very promising, but **BGE-M3 has a broader retrieval-oriented feature story**, especially if you want dense+sparse evolution from the same family.ollama+3

So my recommendation here is very concrete:

- **If you want a strong multilingual embedding candidate with Italian support and enterprise-friendly packaging, Granite Embedding 278M Multilingual should be on your shortlist.**ibm+1
- **If you already know you want advanced hybrid retrieval and very broad multilingual coverage, BGE-M3 still deserves a direct head-to-head comparison.**ollama+1

- **4) Granite Vision is genuinely relevant to your wiki-image requirement**

Your wiki contains **technical images**, including charts, diagrams, and document-like content. Granite Vision is specifically positioned for **visual document understanding**, including **tables, charts, diagrams, sketches, and infographics**, and IBM's Vision docs explicitly mention **chart analysis** and enterprise content extraction. The Hugging Face model card for Granite Vision 3.3 2B likewise describes it as a compact vision-language model for **document understanding**, **OCR**, and **chart/table analysis**.ibm+1

That is a very good match for your chart-heavy requirement. Your attached example — a time-versus-power line chart — is exactly the kind of artifact for which a document-oriented vision model could be useful. Granite Vision 3.3 is also small enough to be interesting for local experiments, while IBM documents strong performance on visual document benchmarks and OCR-oriented evaluations.ibm+1

However, there is one important caveat: the Granite Vision 3.3 model card states that **the model currently supports English instructions and images as input format**. That is not a problem if you use it in the ingestion pipeline with internal English prompts and then store the extracted facts for later bilingual retrieval. But it **does** mean I would not treat Granite Vision as the obvious choice for direct end-user bilingual multimodal chatting without testing alternatives.huggingface

This is where I'd be pragmatic:

- for **offline image-to-text extraction during ingestion**, Granite Vision looks very relevant,
- but for maximum flexibility, I would benchmark it against another strong VLM such as **Qwen2.5-VL**, which is documented as a multimodal model with strong document/chart understanding and support for multilingual text inside images.ibm+3

- **5) Granite Docling may be a hidden gem for your ingestion pipeline**

Even though your source is MediaWiki rather than PDFs, **Granite Docling** is still interesting because IBM positions it as a compact model for **document understanding and conversion**, preserving structure such as **tables, code**

blocks, math, and document hierarchy, and explicitly says the outputs are well suited for downstream tasks such as **RAG**.ibm

If some of your wiki pages or attachments contain exported documents, screenshots, scanned manuals, or technical PDFs, Docling could become very useful as a **pre-processing stage** before chunking and embedding. That is not your first priority, but it fits nicely into a second or third PoC iteration.ibm

---

- **6) Granite Guardian is very interesting for your "knowledge-base-only" requirement**

You said the system should answer from the knowledge base only. No model can guarantee that perfectly, but Granite Guardian is one of the more interesting model-family-level features here. IBM's Guardian docs describe the family as models that can judge risks such as **hallucinations related to RAG**, and specifically assess **context relevance**, **groundedness**, and **answer relevance** in RAG systems. IBM also provides a dedicated **factuality detection** model for judging whether a response is unfactual relative to a context.ibm+1

That makes Guardian unusually relevant for your future production system. I would not put it in PoC v1, but I would definitely keep it in mind for:

- post-generation screening,
- groundedness checks,
- and evaluation tooling.ibm+1

- **My concrete recommendation for your stack**

If I map Granite to your architecture, my recommendation would be:

- **Strong candidates from Granite**
- **Embedding model: Granite Embedding 278M Multilingual** — very plausible candidate for EN/IT retrieval.ibm+1
- **Vision/image-analysis model: Granite Vision 3.3 2B** — especially for charts, tables, OCR, and technical images during ingestion.ibm+1

- **Document conversion / PDF-like ingestion: Granite Docling** — valuable in later phases if you ingest manuals or scanned docs.ibm
- **Guardrails later: Granite Guardian** — very relevant for groundedness / factuality checks in RAG.ibm+1
- **Conversational model**

For the chatbot itself, I would say:

- **Yes, benchmark Granite 4.0 seriously**, especially if you want efficient local deployment and structured outputs.ibm+2
- But I would still compare it against at least **one or two other open instruction models** before locking the PoC, because the conversational model will be the most visible part of the user experience. Granite looks promising, but your final choice should be data-driven.ibm+1

- **My bottom line**

I like Granite a lot for your project — probably more as a "family to benchmark end-to-end" than as a single automatic winner.ibm+1

If you want the most practical summary:

- **Granite Embedding: yes, strong candidate** for EN/IT retrieval.ibm+1
- **Granite Vision / Docling: very relevant** for your image- and document-heavy knowledge base.ibm+1
- **Granite 4.0 language models: worth serious benchmarking** as the conversational model, especially if you value enterprise-friendly efficiency and local deployment.ibm+2
- **Granite Guardian: excellent future add-on** for groundedness and hallucination detection in RAG.ibm+1

If you want, the next useful step would be one of these:

1. **compare Granite vs Qwen/Llama/BGE-M3 for your exact architecture**,

2. **design a Granite-based PoC stack**, or
3. **define a benchmark matrix for conversational, embedding, and vision models**.

Sources

5 GPT-OSS models

You said:

What do you think about GPT-OSS model? Could it be a good choice for this project?

Copilot said:

Copilot

Yes — **GPT-OSS is absolutely a serious candidate for the *conversational model* in your project**, but with two important caveats: **it is text-only**, and **it is not an embedding model**, so it cannot cover the full stack by itself. In other words, GPT-OSS could be a good choice for the chatbot brain, but you would still need a separate **embedding model** and a separate **image-analysis model** for your wiki ingestion pipeline.openai+2

- **My short verdict**

Yes, GPT-OSS could be a good choice for your project — specifically as the conversational model — but not as the only model family.openai+1

More concretely:

- **Good fit as conversational model:** yes, especially if you want an **open-weight**, **Apache 2.0 licensed**, **self-hostable** reasoning model with strong **tool use**, **structured outputs**, and **OpenAI-style compatibility through self-hosted runtimes**.openai+2
- **Good fit as embedding model:** no — GPT-OSS is a **text-only reasoning/chat model**, not an embedding model.openai+1
- **Good fit for image understanding:** no — GPT-OSS is **text-only**, so it is not the right model for analyzing charts, diagrams, and technical images from your MediaWiki knowledge base.openai+1

- **Why GPT-OSS is attractive for your use case**
- **1) It matches your "open-weight + local deployment" requirement very well**

OpenAI's GPT-OSS family consists of **open-weight reasoning models** released under the **Apache 2.0 license**, and OpenAI explicitly says they are meant to run on infrastructure **you control**, including **on-premises** or in a **private cloud**. OpenAI's help center also states that these models are **not served through the OpenAI API** and are intended to be run through open inference stacks such as **vLLM**, **Ollama**, and **llama.cpp**.openai+2

That is a very good fit for your eventual **fully local confidential deployment**.openai+1

- **2) GPT-OSS is designed for reasoning, tool use, and agentic workflows**

The official model card says GPT-OSS models are designed for **agentic workflows**, with strong **instruction following**, **tool use** such as **web search** and **Python code execution**, **structured outputs**, and configurable **reasoning effort**. OpenAI's launch materials also describe the models as strong on reasoning, few-shot function calling, and tool use.openai+2

For your RAG chatbot, those are very relevant strengths because a good conversational model needs to:

- synthesize retrieved context accurately,
- follow grounding instructions strictly,
- cite sources cleanly,
- and optionally return structured answers or metadata.openai+1

- **3) It can be served through an OpenAI-compatible interface**

This is one of the strongest points for your architecture.

OpenAI's own cookbook shows how to run GPT-OSS with **vLLM**, and specifically says vLLM can expose GPT-OSS through **OpenAI-compatible Chat Completions** and **Responses** APIs, so your application can continue using the standard OpenAI SDK pattern by just changing the base_url.openai

That aligns very closely with your requirement that **models should be accessed via OpenAI API so that if you replace them the software interface does not change**.openai+1

- **Why GPT-OSS is not enough by itself for your project**
- **1) It is not an embedding model**

The official GPT-OSS materials present GPT-OSS as a family of **open-weight reasoning models** for text generation, tool use, and structured outputs. The official model card and related documentation describe the models as **text-only** LLMs; they do **not** position GPT-OSS as a text-embedding model.openai+1

Since your architecture explicitly requires:

- one **conversational model**, and
- one **embedding model**,

GPT-OSS can only cover the first role. You would still need something like:

- **BGE-M3**, or
- **Granite Embedding 278M Multilingual**,

for vectorization and retrieval.github+3

- **2) It is text-only, so it does not solve your image-analysis requirement**

Your knowledge base includes **text and images**, and you specifically want images to be analyzed deeply because they may contain critical technical data such as **charts of electrical measurements**.

The GPT-OSS model card states that GPT-OSS models are **text-only models**, and NVIDIA's model overview for GPT-OSS likewise describes GPT-OSS as a **text-only** LLM.openai+1

That means GPT-OSS is **not** the right model for:

- OCR-like extraction from screenshots,

- chart reading,
- table extraction from images,
- diagram interpretation.openai+1

For that part, you still need a **vision/document model**, for example:

- **Granite Vision**, which IBM explicitly positions for **OCR, chart analysis, tables, diagrams, and document understanding**, or
- another strong VLM like **Qwen2.5-VL**.qdrant+3

- **How GPT-OSS maps to your PoC and production needs**
- **PoC**

For a PoC, **gpt-oss-20b** is particularly interesting because OpenAI describes it as the smaller, lower-latency model for **local or specialized use cases**, and the Hugging Face model card says it can run within roughly **16 GB of memory** thanks to the model's quantization approach. OpenAI's documentation also presents the 20B model as suitable for local inference and rapid iteration.github+2

That makes it a plausible PoC candidate if you want:

- local experimentation,
- a reasoning-capable model,
- OpenAI-style serving through vLLM,
- and relatively manageable hardware requirements.openai+1

- **Production**

For production, **gpt-oss-120b** is more interesting if you want stronger reasoning quality. OpenAI says the 120B model is aimed at **production, general-purpose, high-reasoning use cases**, and the official materials say it can fit into a single **80 GB GPU** such as an H100 or MI300X.github+2

So if your future on-prem environment includes serious GPU resources, GPT-OSS 120B becomes much more compelling.github+1

- **The biggest caveats I would watch carefully**
- **1) Harmony response format is not optional**

OpenAI's GitHub repo and Hugging Face model card both say GPT-OSS models were trained using the **Harmony response format** and **should only be used with this format**, otherwise they will not work correctly.github+1

This is not a deal-breaker, but it **does** mean:

- you should use a runtime/tooling stack that already supports GPT-OSS correctly,
- and you should avoid treating GPT-OSS like a generic drop-in raw checkpoint without the proper serving/template layer.github+1

This is one reason I would pair GPT-OSS with **vLLM** rather than building a custom raw inference wrapper.openai+1

- **2) Full chain-of-thought is a power feature — but not for end users**

OpenAI highlights that GPT-OSS provides **full chain-of-thought access**, but also says it is **not intended to be shown to end users**.github+2

For your internal company chatbot, that implies a very practical rule:

- use the chain-of-thought internally for debugging or development if needed,
- but do **not** expose it in the UI.github+1

- **3) Safety and grounding still require application-level controls**

OpenAI's model card explicitly says that open models have a different risk profile than API-hosted models, and that developers/enterprises may need to implement **extra safeguards** to replicate the system-level protections available in managed deployments.openai

That matters for your requirement that the chatbot should answer **from the knowledge base only**. GPT-OSS may be a strong reasoning model, but you still need:

- retrieval grounding,
- citations,
- answer refusal when context is missing,
- and possibly post-generation checks.openai+1

So even if you choose GPT-OSS, the "knowledge-base-only" behavior still has to be enforced at the **RAG application level**, not just by trusting the model.openai

---

- **4) English/Italian support needs benchmarking**

This is the one point where I would be careful.

OpenAI's official GPT-OSS pages strongly emphasize reasoning, tool use, and agentic workflows, but the official materials I found do **not** provide a clear supported-languages list like IBM Granite does for its language and embedding models.openai+3

Because your users must be able to chat in **English or Italian**, I would **not assume** GPT-OSS is automatically your best multilingual option. Instead, I would benchmark it specifically on:

- English technical QA,
- Italian technical QA,
- bilingual paraphrase retrieval,
- and terminology-heavy electronics questions.

That is especially important because some model families, like Granite, explicitly list **Italian** among their supported languages, while GPT-OSS official docs do not make that as explicit.github+3

- **My practical recommendation**
- **Would I include GPT-OSS in your benchmark shortlist?**

Definitely yes.openai+1

- **Would I make it the only planned model family?**

No.openai+1

- **Best role for GPT-OSS in your architecture**

Use GPT-OSS as a **candidate conversational model** in a stack like this:

- **Conversational model:** gpt-oss-20b for PoC, and possibly gpt-oss-120b for stronger production-grade reasoning if hardware allows.github+2
- **Embedding model:** something else, such as **BGE-M3** or **Granite Embedding 278M Multilingual**.github+3
- **Image analysis model:** something else, such as **Granite Vision** or another document-oriented VLM.qdrant+1
- **Serving layer: vLLM**, because OpenAI's own cookbook documents GPT-OSS with vLLM and OpenAI-compatible APIs.openai+1

- **Final answer**

Yes — GPT-OSS could be a very good conversational-model choice for this project, especially if you want an open-weight, locally deployable reasoning model with strong tool-use and OpenAI-compatible serving via vLLM.openai+2

But I would frame it this way:

- **As the chatbot model: good candidate**.openai+1
- **As the full solution: not enough**, because you still need an embedding model and a vision/image model.openai+2
- **For English + Italian: benchmark carefully before committing**, because the official GPT-OSS materials do not spell out supported languages as clearly as some competing families do.openai+3

If you want, the next most useful step would be one of these:

1. **compare GPT-OSS vs Granite 4.0 vs Qwen for the conversational role**,

2. **design a PoC stack using GPT-OSS + BGE-M3 + Granite Vision**, or
3. **build a benchmark plan to test English/Italian technical QA across candidate models.**

Sources

- **March 15**

---

6 Local (static) website mirror

You said:

The local website mirror was created with this command, so it is a static snapshot captured a few months ago:wget --mirror --convert-links --page-requisites --no-parent https://wiki.dave.euAs such, it is not possible to use Mediawiki API to interact with it.

Copilot said:

Copilot

Yes — that changes the **ingestion method**, but **not** the overall feasibility of the project.

In fact, for a PoC, a **static mirrored snapshot is often a very good source format**:

- it is **deterministic**,
- it is **reproducible**,
- it avoids hitting the live wiki,
- and it is easy to version and re-index.

So the right adjustment is:

Switch from "MediaWiki API ingestion" to "filesystem + HTML parsing ingestion."

That is **not a problem** for the architecture.

- **Revised recommendation: ingest the local static mirror directly**

Since your mirror was created with:

Shell

you now have:

- HTML pages
- downloaded images

- CSS / JS / auxiliary assets
- rewritten local links

So your ingestion pipeline should operate on the **local directory tree**, not on MediaWiki endpoints.

- **What changes architecturally**
- **Before**

I had suggested:

- query MediaWiki pages via API
- extract revisions / parsed content / image references
- **Now**

The revised ingestion flow becomes:

1. **walk the mirrored filesystem**
2. **identify actual HTML content pages**
3. **parse the HTML**
4. **extract the main content area**
5. **extract headings, paragraphs, lists, tables, code blocks**
6. **extract image references and associate them with nearby textual context**
7. **process local image files**
8. **chunk + embed + index**

So only the **source connector** changes.
The rest of the RAG stack remains the same.

- **This is actually a good fit for your PoC**

A static snapshot has several advantages for experimentation:

- **Advantages**
- **Stable corpus**: every retrieval test is reproducible
- **Easy benchmarking**: same corpus for all model/retrieval comparisons
- **No authentication/API complexity**
- **No dependence on the live wiki structure**
- **Easy cloud experimentation** because the corpus is just a folder
- **Limitations**
- no revision history
- no live page metadata from the wiki backend
- categories/templates may be harder to reconstruct semantically
- some pages may include duplicated navigation or boilerplate
- some useful metadata that exists in the live wiki may no longer be conveniently structured

These are manageable.

- **Revised ingestion strategy for a static MediaWiki mirror**
- **1) Treat the mirror as a local website, not as a document dump**

Your ingestion pipeline should include a **site parser** layer with these responsibilities:

- **A. Discover relevant pages**

Scan the mirror directory and include only:

- .html
- .htm
- maybe index.html under page folders
- pages corresponding to wiki content

Exclude:

- CSS
- JS
- fonts
- thumbnails without context pages
- login/help/system pages unless you really want them
- duplicate redirect pages if present
- **B. Canonicalize pages**

Because wget --convert-links rewrites paths, the same wiki page may appear in filesystem-friendly form.

You should create a **canonical document ID** for each page, for example from:

- original URL if recoverable
- canonical link tag in HTML if present
- page title + relative local path if not

I strongly recommend storing both:

- local_path
- original_url (if reconstructable)

- **2) Parse the main wiki content from HTML**

For MediaWiki-derived HTML, you usually want to extract only the **main content area**, not the full page.

Typical targets often include elements like:

- page title area
- main content container
- body text container
- headings
- tables

- image blocks
- category/footer metadata

The exact CSS selectors may vary depending on the site theme, so I would implement this robustly:

- **Extraction strategy**

1. Try site-specific selectors first
   for example:
   - #content
   - #mw-content-text
   - main
   - .mw-parser-output
2. Fall back to generic content extraction if needed
3. Remove boilerplate:
   - navigation
   - sidebar
   - footer
   - edit links
   - breadcrumbs
   - search boxes
   - cookie banners
   - scripts/styles

- **Why this matters**

If you embed the full page HTML without cleaning, retrieval quality will degrade because chunks become polluted by:

- menus
- repeated headers

- unrelated navigation text
- boilerplate links

- **3) Preserve document structure during extraction**

This is **very important** for technical documentation.

For each page, preserve:

- page title
- section hierarchy (H1/H2/H3/...)
- paragraphs
- bullet lists
- numbered procedures
- tables
- code/preformatted blocks
- image references
- captions / nearby explanatory text

This allows you to build **section-aware chunks**, which work much better than naive chunking for technical wikis.

- **Images become even more important in a static mirror**

Because you cannot query image metadata from the MediaWiki backend, you should recover image meaning from the HTML context plus the image itself.

- **For each image, capture:**
- **From HTML context**
- image file path
- src
- alt text if present

- figure caption if present
- nearest heading
- nearest paragraph before and after
- table cell / infobox context if the image is embedded there
- page title + section title
- **From image analysis**
- OCR text
- detailed image description
- chart/table/diagram interpretation
- structured summary

This is exactly where your requirement about technical images becomes central.

- **Recommended static-mirror ingestion pipeline**

Here is the revised pipeline I recommend.

- **Stage 1 — File discovery**

Build a manifest like:

Plain Text

- **Stage 2 — HTML parsing**

For each HTML page:

- extract clean text content
- preserve structure
- identify content blocks
- identify images and nearby text
- **Stage 3 — Content normalization**

Normalize:

- whitespace
- HTML entities
- duplicated boilerplate
- table text
- code blocks
- internal anchor links
- **Stage 4 — Image enrichment**

For each referenced local image:

- load local file
- OCR
- VLM analysis
- generate technical textual summary
- optionally JSON summary for charts/tables
- **Stage 5 — Chunking**

Create chunks such as:

- section text chunks
- table summary chunks
- image summary chunks
- mixed contextual chunks (optional)
- **Stage 6 — Embedding + indexing**

Embed all chunks and store them in your vector DB with payload metadata.

- **Recommended metadata schema for static mirror ingestion**

Because you lose API-level metadata, metadata design becomes even more important.

For each chunk, I recommend storing:

JSON

This will help a lot later for:

- debugging,
- filtering,
- evaluation,
- citation rendering.

- **How to handle charts in the static mirror**

Your attached example is exactly the kind of asset that the system must not ignore.

For a chart image from the local mirror, your VLM/image pipeline should produce output like:

- **Natural-language summary**
- chart type: line chart
- x-axis: Time [s], approx. 0–180
- y-axis: SoM Power [W], approx. 0.0–3.0
- trace mostly stable around ~1.65–1.72 W
- multiple short peaks up to about ~2.2 W
- increased spike activity around ~100–120 s and near the end
- **Structured summary**

For example:

JSON

Then embed:

- the natural-language summary
- the structured JSON serialized as text

- the surrounding page text

This makes the image retrievable for queries like:

- “What is the idle power?”
- “Are there spikes in the power measurement?”
- “Show charts about SoM power over time”
- “What is the approximate peak power?”

- **Revised recommendation for the PoC stack**

Now that the source is a static snapshot, I would recommend this stack even more strongly:

- **Ingestion**
- **Python**
- BeautifulSoup / lxml for HTML parsing
- custom site parser for MediaWiki-like pages
- local file handling for images
- **Embeddings**
- **BGE-M3** or **Granite Embedding 278M Multilingual**
- both are suitable candidates for EN/IT retrieval
- **Conversational model**
- still benchmark candidates such as:
    - GPT-OSS
    - Granite 4.0
    - Qwen-family instruct models
- **Image analysis**
- local VLM such as:

    - Granite Vision
    - or another strong open VLM
- optionally OCR layer
- **Retrieval store**
- Qdrant or Weaviate
- **UI**
- Streamlit
- **Serving layer**
- vLLM (for conversational model and possibly embedding model, depending on what you standardize)

- **Important implication for the future production system**

This clarification also suggests something useful for production:

You do **not** need to ingest directly from the live confidential MediaWiki.

A very robust production pattern would be:

1. periodically create a **local static mirror/export** of the private wiki
2. run ingestion from that mirror
3. re-index incrementally or fully
4. keep the chatbot completely decoupled from the live documentation server

That has real advantages:

- less load on the documentation platform
- easier reproducibility
- easier change control
- safer indexing pipeline
- easier rollback to prior snapshots

So the static-snapshot approach is not just a PoC workaround — it could actually become part of the production architecture.

- **My revised recommendation in one sentence**

Use a filesystem-based HTML-and-image ingestion pipeline over the static wget snapshot, with section-aware parsing and first-class image enrichment.

That is fully compatible with your final goal.

- **What I would do next**

The most useful next step now is **not** choosing more models.
It is to define the **ingestion contract** for the static mirror.

I suggest the next deliverable should be one of these:

- **Option 1 — Best next step**

A detailed design of the ingestion pipeline for the static mirror

- page discovery
- HTML parsing rules
- chunk schema
- image-context association rules
- metadata schema

- **Option 2**

A Python project skeleton for:

- crawler/indexer
- parser
- image enrichment
- embedding/indexing

- Streamlit chat app
- **Option 3**

A first implementation plan with milestones:

- v0.1 dense retrieval
- v0.2 image-aware ingestion
- v0.3 hybrid search
- v0.4 evaluation framework

If you want, I can now do the most practical next step:

Design the static-mirror ingestion pipeline in detail, including folder structure, parsing rules, and chunk schema.

7 Different approaches for mirroring the test website

You said:

For the purpose of this project, do you think I should recreate the local mirror using a different, more convenient approach?

Copilot said:

Copilot

Yes — **if you can reacquire the corpus, I would recommend using a different acquisition strategy** for this project. But I would be precise about *what* to change:

I would not primarily switch from wget to another mirroring tool like HTTrack.
I would switch from "HTML snapshot as the primary source" to "structured wiki export as the primary source," and keep an HTML/assets mirror only as a secondary artifact.mediawiki+2

That is the key distinction.

- **Short answer**
- **My recommendation**
- **For the current PoC:** your existing wget mirror is **usable**, so I would **not block progress** waiting for a better mirror.
- **For a cleaner and more maintainable pipeline:** yes, **recreate the dataset using a more convenient approach if possible** — ideally a **MediaWiki XML export/dump + separate image/file capture**, not just another static HTML scraper.mediawiki+1

- **Why I think your current wget mirror is *usable but suboptimal***

A website mirror created with recursive mirroring tools is good for offline browsing because it preserves HTML pages, images, and relative links locally. HTTrack's official documentation explicitly describes this kind of output: it recursively downloads HTML, images, and other files, preserves relative link structure, and lets you browse the mirrored site offline like the original. That is essentially the same class of artifact you already created with wget --mirror --convert-links --page-requisites --no-parent.httrack

So from a **website archiving** perspective, your current snapshot is perfectly valid. But from a **RAG ingestion** perspective, static HTML has disadvantages:

- it mixes content with navigation and theme boilerplate,
- it loses clean access to underlying wiki markup,
- it usually loses structured revision/export semantics,
- and it makes page/image association and metadata extraction more fragile than necessary.
  Those drawbacks are exactly why MediaWiki offers export mechanisms such as **Special:Export** and **XML dumps**.mediawiki+1

So the problem is **not** that wget is "wrong."
The problem is that **HTML is not the most convenient canonical ingestion format for a technical RAG pipeline**.mediawiki+1

- **The key point: switching from wget to HTTrack does not solve the main issue**

If you are asking whether you should recreate the mirror using **another mirroring tool**, such as HTTrack, my answer is:

Not really — not as the main improvement.

HTTrack is a legitimate offline website copier and supports recursive mirroring, link rewriting, project updates, and resume capabilities. But it still produces a **mirrored website**, i.e. mostly **rendered HTML plus assets**, not a clean wiki-content export.httrack+1

So changing:

- from wget HTML mirror
  to
- HTTrack HTML mirror

may improve some crawling convenience, but it **does not fundamentally fix** the RAG-ingestion issue. You would still have to parse HTML, remove boilerplate, reconstruct structure, and recover image context from rendered pages.httrack+1

That is why I would not invest effort in "mirror tool swapping" unless:

- the current mirror is incomplete,
- assets are missing,
- local links are broken,
- or you need better crawl filtering.

Otherwise, the bigger win is changing the **source format**, not the **mirroring utility**.httrack+2

- **What I would do instead: a two-artifact acquisition strategy**

For this project, the best setup is usually:

- **1) Structured content export as the main ingestion source**

Use one of the native MediaWiki export mechanisms:

- **Special:Export** for web-accessible XML export of pages, or
- **dumpBackup.php** if you control the MediaWiki server and can run maintenance scripts.mediawiki+1

MediaWiki's official documentation states that **Special:Export** can export pages in a special **XML format**, optionally including all revisions and templates. It also states that **dumpBackup.php** creates an XML dump of wiki pages and revisions, and can optionally include uploaded files.mediawiki+1

- **2) Static HTML/assets snapshot as a secondary artifact**

Keep or regenerate an HTML mirror for:

- visual inspection,
- source citation links,
- debugging,
- and image file capture if your export path does not include everything you need.httrack+1

This combination gives you:

- **clean text ingestion** from XML/wikitext,
- **faithful rendered context** from HTML,
- and **local images/assets** for multimodal enrichment.mediawiki+1

- **Best options, ranked**
- **Option A — Best overall: MediaWiki XML export + asset mirror**

If the source wiki is reachable and allows export, this is the best balance.

- **Why it is best**
- XML export is purpose-built for wiki content extraction.mediawiki+1
- It preserves page content more cleanly than HTML scraping.mediawiki
- It gives you a much better base for section-aware parsing, template handling, and metadata extraction.mediawiki+1
- You can separately mirror/download images and uploaded files. dumpBackup.php can even include uploaded files when run with the appropriate options.mediawiki
- **When to choose it**
- For the **production confidential wiki**, especially if you control the MediaWiki instance or can ask the admin team for exports.mediawiki+1
- For a **cleaner PoC refresh**, if reacquiring from the live public wiki is acceptable.mediawiki+1

- **Option B — Good enough for now: keep the current wget mirror**

This is what I would do **if speed matters more than perfect ingestion cleanliness right now**.

- **Why it is acceptable**
- You already have the snapshot.
- It is reproducible.
- It can absolutely be parsed locally.
- It already contains the rendered image/page relationships you need for the first PoC.

This is especially reasonable if your immediate goal is:

- validate chunking,
- test embeddings,
- prototype image analysis,
- and get the first RAG chatbot running.
- **When to choose it**
- If you want to start coding **immediately**.
- If reacquiring the source would take time or require permissions.
- If the current mirror is materially complete.

In that case, I would **not** recreate the mirror first. I would instead build the ingestion pipeline and keep "better corpus acquisition" as the next improvement. This is my engineering recommendation based on effort vs payoff, not a claim from the web sources.

- **Option C — Recreate the HTML mirror with better filters**

If you cannot get XML export and must stay with HTML, then yes, I would consider recreating the mirror — **but only to make it cleaner**, not because another mirror tool is fundamentally better.

- **Improvements worth making**
- exclude Special: pages,
- exclude help/system pages you don't need,
- avoid edit/history/raw/action pages,
- constrain namespaces/categories if possible,
- and ensure all image assets are captured.

This won't be as good as a structured export, but it can reduce ingestion noise substantially.
The general idea of filtering unwanted namespaces/actions is consistent with long-standing MediaWiki static-site practices described in community examples, although the official MediaWiki docs themselves recommend export-oriented methods for structured content capture.mediawiki+1

- **What about the public DAVE wiki specifically?**

One important nuance: although **your local snapshot** cannot use the MediaWiki API, the **live DAVE wiki** does appear to expose a MediaWiki API endpoint (api.php). That means the *website itself* is still a MediaWiki installation with API surface; your limitation is specifically the **offline mirror format**, not MediaWiki in general.dave+1

So if reacquiring from the public site is acceptable for PoC purposes, you could move to a better format now.
For the confidential production wiki, if it is also MediaWiki-based, I would strongly prefer a structured export or server-side dump over HTML mirroring whenever possible.mediawiki+2

- **My concrete recommendation for your project**
- **If I were designing this with you, I would choose:**
- **PoC now**

- **Keep the current wget snapshot** and start building the ingestion pipeline immediately.
- Do **not** delay the PoC just to replace wget with HTTrack.
  That tool change alone is not the main architectural improvement.httrack+1
- **PoC v2 / corpus refresh**
- Reacquire the corpus in a more convenient format:
  - **MediaWiki XML export** if possible, and
  - a **separate image/assets mirror**.mediawiki+1
- **Production**
- If you control the confidential MediaWiki:
  - use **dumpBackup.php** or another admin-controlled export path as the primary content source,
  - and keep a local asset copy for images/files.mediawiki

- **Final verdict**
- **Should you recreate the local mirror using a different, more convenient approach?**

Yes — if you mean moving to a structured export workflow.mediawiki+1

- **Should you recreate it just by swapping wget for HTTrack?**

Probably no — not unless the current mirror is incomplete or messy. Both tools fundamentally create the same kind of artifact: a browsable offline HTML mirror.httrack+1

- **Best practical decision right now**
- **Do not throw away the current mirror.**
- **Start with it for PoC v1.**
- **Plan a better acquisition pipeline for v2**, ideally:
  - **MediaWiki XML export as canonical text source**

- **local media/assets mirror as canonical image source**.mediawiki+1

If you want, I can now help with one of these very practical next steps:

1. **design the ideal acquisition strategy for PoC and production**,
2. **propose the exact folder structure for XML export + asset mirror ingestion**, or
3. **draft a migration plan from the current wget snapshot to a cleaner source format**.

Sources

8 Managing live changes of the knowledge

You said:

When I move to the production deployment, the RAG system will interact with the confidential wiki via API. The confidential wiki is dynamic, in the sense that contents change over time because of users' editing activity. Is it possible to capture these events asynchronously? How to avoid data duplication? For instance, if user A modifies page P, the RAG system should update the knowledge base accordingly: all the information related to the old version of P should be replaced with the new ones.

Copilot said:

Copilot

Yes — **this is absolutely possible**, and for a production RAG system connected to a live MediaWiki instance, it is actually the **right** way to operate: capture page-change events asynchronously, enqueue re-indexing work, and make the vector store reflect the **current revision** of each page rather than accumulating stale versions. MediaWiki provides both **pull-style change discovery** through list=recentchanges and **push-style integration points** through hooks and recent-changes feeds, so you can implement this cleanly.mediawiki+3

- **Short answer**
- **Yes, you can capture changes asynchronously**

You have **two viable patterns**:

1. **Push model (recommended if you control the confidential wiki):** use MediaWiki hooks such as PageSaveComplete, PageMoveComplete, PageDeleteComplete, or RecentChange_save, and publish a message to your own queue/webhook/Redis topic whenever a page changes. MediaWiki also supports machine-readable recent-changes feeds via $wgRCFeeds and built-in formatters such as JSONRCFeedFormatter, with engines such as Redis Pub/Sub and UDP.mediawiki+4
2. **Pull model (portable fallback):** poll the MediaWiki Action API using list=recentchanges, which can return timestamps, page IDs, titles, and the **old/new revision IDs** for edits.mediawiki

- **Yes, you can avoid data duplication**

The clean pattern is:

- treat **page_id** as the stable logical document identity,
- treat **rev_id** as the immutable content version,
- rebuild the page's chunks for the new revision,
- then atomically mark the new revision as **current** and delete or deactivate the old revision's chunks. MediaWiki's revisions API exposes revision IDs, timestamps, and content hashes (sha1), which are useful for idempotency and change detection.mediawiki

---

- **Recommended production architecture**

I recommend an **event-driven ingestion architecture** with four layers:

1. **MediaWiki change capture**
2. **Queue / event bus**
3. **Ingestion worker**
4. **Atomic index replacement**

This keeps user edits on the wiki fast and decouples them from heavier work such as parsing, OCR, image analysis, embedding generation, and vector upserts. MediaWiki's hook system exists specifically so extensions can react to wiki events, and several hooks fire after page saves, moves, and deletes.github+3

- **Best option: push-based asynchronous capture**

If you control the confidential wiki, this is the best design.

- **Hooks you can use**
- **PageSaveComplete** fires **after an article has been updated** and provides the RevisionRecord of the new article revision.mediawiki+1

- **RecentChange_save** fires at the end of RecentChange::save(), which is useful if you want to react directly to the recent-changes record that MediaWiki writes.wikimedia
- **PageMoveComplete** fires **after an article has been moved**, post-commit.mediawiki+1
- **PageDeleteComplete** fires **after a page is deleted**.mediawiki+1
- **What the hook handler should do**

The hook handler should **not** do full RAG ingestion inline. Instead, it should publish a small event to your internal queue, for example:

JSON

This is the safest pattern because MediaWiki hooks are extension points, and heavy processing directly in the request path would increase latency and operational risk. Hooks are intended to let extensions respond to wiki events, not to embed a full indexing pipeline into the edit request itself.github+3

- **Alternative push option: recent-changes feed / stream**

If you prefer not to write a custom hook extension, MediaWiki can also expose recent changes as a machine-readable feed.

- MediaWiki documents a **machine-readable Recent Changes feed schema** (Manual:RCFeed). The feed includes fields such as event ID, type, namespace, title, timestamp, user, revision old/new IDs, and log-event details.mediawiki
- MediaWiki's $wgRCFeeds configuration allows sending recent-changes network updates to configured destinations **after** the change has been inserted into the recentchanges table, and it supports formatters including **JSONRCFeedFormatter** and engines including **Redis Pub/Sub** and **UDP**.mediawiki
- The MediaWiki "Recent changes stream" documentation also explains that if you run your own wiki, you can configure similar feeds and expose changes

over HTTP, WebSockets, IRC, or other channels; alternatively, you can always request changes through the regular API in a pull model.mediawiki

For a private corporate network, **JSON recent-changes feed → internal queue consumer** is a very clean architecture.mediawiki+1

- **Portable fallback: pull from the API**

If you do not want to modify the confidential wiki at first, you can still capture updates asynchronously by polling list=recentchanges.

The official API states that list=recentchanges enumerates recent changes, and rcprop can include:

- **timestamp**
- **title**
- **page ID**
- **recent changes ID**
- **new and old revision IDs**
- **new and old page length**
- redirect flag and other metadata.mediawiki

That means a polling worker can:

1. keep a **watermark** (last processed timestamp and/or RC ID),
2. periodically call query&list=recentchanges,
3. extract changed page IDs and revision IDs,
4. enqueue only unseen events.mediawiki

This is less immediate than hooks, but it is simpler to roll out because it requires no wiki extension.mediawiki+1

- **How to fetch the updated canonical content**

Once an event is captured, the ingestion worker should fetch the **canonical new revision** before re-indexing.

- **Use the revisions API for source content and revision metadata**

prop=revisions can return:

- revision **IDs**
- **timestamp**
- **size**
- **sha1**
- **content model**
- and the actual **content** of the revision.mediawiki

This is the best basis for version-aware ingestion.

- **Use the parse API when you need rendered structure**

If your indexing pipeline needs section lists, rendered HTML, or image references tied to a specific revision, action=parse can operate on a page or on a specific **oldid** and can return:

- parsed text,
- sections,
- images,
- links,
- templates,
- categories,
- display title,
- and the revision ID of the parsed page.w3

That is very useful if your chunking pipeline depends on rendered structure or on extracting page-image associations consistently for the **current revision**.w3

- **How to avoid duplication and stale content**

The key is to separate **logical identity** from **version identity**.

- **Recommended identities**

- **1) Stable page key**

Use **page_id** as the stable logical document identity. recentchanges and the revisions API both expose page IDs and revision IDs, so you do not need to rely on titles as the primary key. Titles can change on page moves, while page identity remains stable through the page lifecycle.mediawiki+1

- **2) Immutable revision key**

Use **rev_id** as the immutable content version key. The recent-changes API exposes old/new revision IDs for edits, and the revisions API can retrieve revision data by revision ID.mediawiki+1

- **3) Chunk key**

For each chunk, store something like:

- page_id
- rev_id
- chunk_index or chunk_hash
- chunk_type (text, image_summary, table_summary, etc.)

This makes every chunk versioned and traceable back to a specific page revision. The revisions API's sha1 field is particularly useful if you want to detect repeated content and skip work on null/no-op changes.mediawiki+1

- **Recommended replacement algorithm**

For a page update, do **not** mutate chunks in place one by one. Instead, use a **staging-and-swap** approach.

- **Algorithm**

1. Receive event for page_id = P, new_rev_id = R2, possibly old_rev_id = R1. list=recentchanges can provide both old and new revision IDs for edits.mediawiki
2. Fetch revision R2 via prop=revisions.mediawiki
3. Parse/chunk/OCR/image-analyze/embed the entire page revision R2. If you need rendered sections/images, use action=parse with oldid=R2.mediawiki+1
4. Write all new chunks for (page_id=P, rev_id=R2) into the vector store / metadata store in a **staging** state.
5. In one atomic metadata transaction, update the page manifest so that current_rev_id = R2 for page P.
6. Delete or mark inactive all chunks for (page_id=P, rev_id=R1) after the swap succeeds.

This guarantees that search results always point either to the full old version or the full new version, never to a half-updated mixture. The reason this works well with MediaWiki is that revisions are explicit first-class objects with IDs, timestamps, and content hashes.mediawiki+1

- **Why full-page reindexing is the best starting point**

It is technically possible to compare revisions and try to update only affected chunks:

- MediaWiki provides action=compare for diffing two revisions, and the REST compare endpoint can also return structured line-by-line comparisons.mediawiki+1

However, for a production RAG system I would start with:

Re-index the whole page on every content change.

Why?

- It is simpler.
- It avoids “orphan stale chunk” bugs.

- It handles structural changes (headings, tables, images, moved text) more safely.
- It is easier to reason about operationally.

Later, if re-indexing throughput becomes a bottleneck, you can add revision diffing to optimize only unchanged sections. MediaWiki's compare APIs make that possible, but it is an optimization, not the best v1 design.mediawiki+1

- **Handling moves and deletes correctly**

This is important for avoiding duplication.

- **Page moves**

A title move should **not** create a second logical document. PageMoveComplete fires after a move and provides the old title, new title, page ID, redirect ID, reason, and revision information. That lets you update the page's canonical title metadata while preserving the same logical page identity.mediawiki+1

- **Page deletes**

A delete event should remove or tombstone the page in the RAG store. PageDeleteComplete fires after a page is deleted and provides the page ID and the last revision record.mediawiki+1

- **Log events**

If you are consuming recent-changes feeds instead of hooks, the machine-readable RC feed schema includes **log events** with log_type, log_action, and log_params, in addition to normal edit/new events. That lets a feed consumer react to more than just edits.mediawiki

---

- **Idempotency and deduplication rules**

To avoid duplicate indexing work and duplicate vectors, implement these rules:

- **1) Queue dedupe**

Use an idempotency key such as:

- revid
- or (page_id, new_rev_id)

Because revision IDs are immutable and unique, they are ideal event identifiers. list=recentchanges provides old/new revision IDs, and the revisions API can retrieve a revision directly by ID.mediawiki+1

- **2) Skip identical content**

Use the revision sha1 returned by the revisions API to detect identical revision content and skip re-indexing if your business rules allow it. MediaWiki explicitly exposes revision SHA-1 in rvprop.mediawiki

- **3) Unique chunk keys**

Enforce a uniqueness rule such as:

- (page_id, rev_id, chunk_hash)
  or
- (page_id, rev_id, chunk_index)

This prevents accidental duplication if a worker retries the same revision. The revisions API gives you the revision identity, and your pipeline can derive deterministic chunk hashes from the normalized chunk text.mediawiki

- **4) Only one active revision per page**

Maintain a manifest table (or equivalent metadata collection) where each page_id has exactly one current_rev_id. Search should filter to only active/current revisions, or old revisions should be physically removed after the swap. This is an application-level design choice built on top of MediaWiki's revision model.mediawiki+1

- **My recommended production pattern**

If I were designing your production deployment, I would choose this:

- **Preferred pattern**

- **MediaWiki extension** using PageSaveComplete, PageMoveComplete, and PageDeleteComplete to emit compact events to an internal queue.mediawiki+2
- **Background worker** that fetches the latest revision via prop=revisions, optionally action=parse, and rebuilds the page's chunk set.mediawiki+1
- **Atomic current-revision swap** in the RAG metadata layer, then deletion/tombstoning of the old revision's chunks.
- **Idempotency on revision ID** and optional sha1 skip logic.mediawiki
- **Good fallback pattern**
- **Polling worker** using list=recentchanges with a timestamp/RC watermark if you do not want to extend MediaWiki initially.mediawiki+1

- **Direct answer to your example**

If user A modifies page P, the RAG system should update the knowledge base accordingly: all the information related to the old version of P should be replaced with the new ones.

Yes — the clean way is:

1. capture the change event for page P,
2. fetch the new revision R2,
3. rebuild all chunks for R2,
4. atomically switch page P's active revision from R1 to R2,
5. remove or deactivate all chunks tied to R1.

That design guarantees no duplication of active content and keeps the vector store synchronized with the current wiki state. MediaWiki's APIs and hooks provide exactly the revision-level metadata needed to implement this safely.mediawiki+4

If you want, my next step can be either:

1. **design the exact event-driven production architecture**,

2. **propose the database schema for page/revision/chunk versioning**, or
3. **draft the update algorithm in Python pseudocode for MediaWiki → queue → re-index → swap**.

Sources

9 Feasibility of local-only PoC

You said:

I would like to run the very first version of the PoC entirely on my local machine. It is equipped with an NVIDIA GeForce RTX 3060 12GB RAM. What models do you suggest to use for this configuration?

Copilot said:

Copilot

Yes — with an **RTX 3060 12GB**, you can absolutely run a **first local-only PoC**, but I would keep the first version **small, quantized, and modular**: one lightweight chat model, one compact multilingual embedding model, and one small vision model for image enrichment. I would also avoid "hero models" like **gpt-oss-20b** on this machine for v1, because OpenAI describes gpt-oss-20b as fitting in **16GB of memory**, not 12GB, and its own positioning is "edge devices with just 16 GB of memory," which is already above your GPU VRAM budget.huggingface+1

- **My recommended first local-only PoC stack**
- **1) Conversational model: Qwen3 8B**

This is my **first choice** for your machine. The Ollama build of qwen3:8b is distributed as a **5.2GB Q4_K_M** quantized model, and Qwen3 explicitly supports **100+ languages and dialects**, with strong multilingual instruction following and translation — which is directly relevant because your users need to chat in **English and Italian**.ollama

- **Why I like it for your RTX 3060 12GB**
- It is small enough to be realistic on your hardware in quantized form. The Ollama package size is **5.2GB**, which makes it much more practical than larger reasoning models.ollama
- It is explicitly multilingual, so it is a safer first pick than models whose Italian support is less clearly documented. Qwen3's model page says it supports **100+ languages and dialects**.ollama

- It is general-purpose and strong enough for a first RAG assistant, without forcing you into a very small model that may underperform on technical QA.ollama

- **When I would choose something else**

If you want to stay entirely within the IBM ecosystem, I would swap this for **Granite 4.0 Micro (3B)** instead of Qwen3 8B. IBM's Granite 4.0 family explicitly supports **Italian**, and the Ollama granite4:3b package is only **2.1GB**, which gives you more GPU headroom and likely lower latency.ollama+1

- **2) Embedding model: Granite Embedding 278M Multilingual** ***(default pick for this PoC)***

For your very first local PoC, I would lean toward **Granite Embedding 278M Multilingual** because it is compact and explicitly supports **Italian** and **English**. IBM's model card says the multilingual Granite Embedding 278M model supports **English, German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, and Chinese**, and the Ollama package for the 278M version is listed at about **563MB**.theverge+1

- **Why this is a good first local choice**

- It is small and lightweight, which is ideal for a single-GPU workstation doing local experimentation. The 278M Ollama package is only about **563MB**.openai

- It covers the languages you care about immediately, including **Italian**.theverge

- IBM positions Granite Embedding specifically for **semantic search and RAG**.ori

- **Alternative: BGE-M3**

If you want to optimize the PoC for future retrieval experiments, then **BGE-M3** is the stronger retrieval-oriented alternative. BGE-M3 supports **100+ languages**, handles inputs up to **8192 tokens**, and supports **dense, sparse, and multi-**

vector retrieval in the same model family. The official docs list it as **569M parameters / 2.27GB**, and the Ollama package is about **1.2GB**.bge-model+1

- **My practical advice**
- **Pick Granite Embedding 278M** if you want the **lightest and simplest** local PoC.theverge+1
- **Pick BGE-M3** if you already know you want to explore **hybrid search** and richer retrieval strategies in PoC v2.huggingface+1

- **3) Image-analysis model: Granite Vision 3.3 2B** ***(default pick for your use case)***

Because your wiki contains technical charts, tables, and screenshots, I would start with **Granite Vision 3.3 2B** for image enrichment. IBM describes Granite Vision as being designed for **visual document understanding**, including **tables, charts, diagrams, sketches, and infographics**, and the Ollama package for ibm/granite3.3-vision:2b is listed at **3.6GB**.ibm+2

- **Why this is a strong fit**
- Your use case is not generic image captioning; it is **technical documentation images**. Granite Vision is explicitly tailored to **OCR, chart analysis, table extraction, and document content understanding**.ibm+1
- The model is small enough to be plausible on your machine in Ollama form: the 2b package is **3.6GB**.ollama+1
- IBM positions it exactly for enterprise document-style content, which is much closer to your wiki than a general image model.ibm+1
- **Alternative: Qwen2.5-VL 3B**

If you want a broader, more general-purpose VLM, qwen2.5vl:3b is also a very reasonable option. The Ollama package is **3.2GB**, and Qwen2.5-VL is explicitly described as being capable of analyzing **texts, charts, icons, graphics, and layouts within images**.ollama+1

- **My practical advice**

For *your* wiki, I would personally start with:

- **Granite Vision 3.3 2B** for the first ingestion prototype, because the model is clearly aimed at **document/chart understanding**.ibm+1
- Keep **Qwen2.5-VL 3B** as your comparison model if you want to benchmark image extraction quality later.ollama+1

- **Recommended baseline combination**

For your RTX 3060 12GB, my favorite **balanced first local PoC** is:

- **Chat:** qwen3:8bollama
- **Embeddings:** granite-embedding:278m *(or BGE-M3 if retrieval experimentation matters more than footprint)*theverge+3
- **Image analysis:** ibm/granite3.3-vision:2bollama+1

That gives you:

- strong multilingual chat for EN/IT,ollama
- a compact multilingual embedding model,theverge+1
- and a document-oriented vision model for charts and OCR-like tasks.ibm+1

- **If you want a single-family IBM stack**

If you prefer maximum coherence and minimum tooling friction, an all-IBM baseline is also very attractive on your hardware:

- **Chat:** granite4:3bollama+1
- **Embeddings:** granite-embedding:278mtheverge+1
- **Image analysis:** ibm/granite3.3-vision:2bollama+1
- **Pros**

- Very lightweight overall. granite4:3b is only **2.1GB** in Ollama form, and Granite 4 explicitly supports **Italian**.ollama
- Strong conceptual consistency across chat, embeddings, and vision. IBM has coherent documentation for all three families.ibm+2
- **Cons**
- For raw conversational quality, I would still expect **Qwen3 8B** to be the stronger first benchmark candidate than Granite 4.0 Micro 3B, simply because you are comparing an 8B multilingual model against a much smaller 3B model. That is my engineering judgment; the size and multilingual scope are documented, but the recommendation itself is my opinion.ollama+1

- **Models I would not use first on this machine**
- **GPT-OSS 20B**

I would **not** choose GPT-OSS 20B for the very first local-only PoC on a 12GB RTX 3060. OpenAI says gpt-oss-20b runs within **16GB of memory**, and their launch post explicitly positions it around a **16GB** hardware envelope. LM Studio also lists the smallest GPT-OSS as needing **at least 12GB of RAM**, but OpenAI's own messaging around gpt-oss-20b still centers on **16GB memory** for local/edge deployment.huggingface+2

That means it may be **possible** with CPU offloading or non-ideal settings, but it is **not the comfortable, low-friction choice** for your first all-local experiment.huggingface+1

- **Large chat models (14B+, 20B+, 30B+)**

I would also avoid:

- Qwen3 14B / 30B,ollama
- Granite 4.0 Small, which LM Studio lists at around **21GB** RAM needs,lmstudio
- and any large VLM beyond the 2B–3B class.ollama+1

For a first PoC, smaller models will keep your latency and operational frustration much lower.

- **One important operational tip**

On a **single 12GB GPU**, I would **not** try to keep all three models resident at once. Instead:

- use the **embedding model** during ingestion/indexing,
- use the **vision model** during image enrichment,
- and use the **chat model** during interactive QA.

In other words, treat the pipeline as **sequential**, not "all models loaded simultaneously." That is the realistic way to make a 12GB card feel usable for a multimodel PoC. This is my deployment advice based on the model footprints above.ollama+2

- **My final recommendation**

If you want the **best first local baseline** for your machine, I suggest:

- **Best mixed stack**
- **Conversational model: Qwen3 8B**ollama
- **Embedding model: Granite Embedding 278M Multilingual**theverge+1
- **Image-analysis model: Granite Vision 3.3 2B**ibm+1
- **Best all-IBM stack**
- **Conversational model: Granite 4.0 Micro 3B**ollama+1
- **Embedding model: Granite Embedding 278M Multilingual**theverge+1
- **Image-analysis model: Granite Vision 3.3 2B**ibm+1
- **Best retrieval-focused alternative**
- swap the embedding model to **BGE-M3** if you want to prepare early for **hybrid retrieval** experiments.huggingface+2

If you want, I can now do one of these next:

1. **propose the exact local PoC stack (models + vector DB + UI + serving runtime) for the RTX 3060**,
2. **suggest whether to use Ollama or vLLM on this machine**, or
3. **give you a phased upgrade path: first local-only on the 3060, then migrate to a stronger server later**.

Sources

🤗 🌀 🦙

10 PoC and remote providers

You said:

Once the very first version of the PoC works locally, I would like to move the models to a more powerful remote server. To this end, I am considering https://www.together.ai/. Are there alternatives that you would suggest?

Copilot said:

Copilot

Yes — **Together AI is a reasonable choice**, but I would not treat it as the only serious option. For your specific roadmap — **start local, then move the PoC to a stronger remote machine, while keeping the software interface stable and still working with open-weight models** — I would suggest evaluating providers in **two different categories**:

1. **Managed inference platforms** (you call an API, they handle serving/scaling), and
2. **GPU infrastructure platforms** (you run your own stack, e.g. vLLM, on rented GPUs).

That distinction matters much more than the brand name.together+2

- **My short recommendation**

If you want the **closest alternatives to Together AI**, I would look first at:

- **Fireworks AI** — very strong if you want **OpenAI-compatible hosted inference** plus the option for **dedicated GPU deployments** and **custom models** later.fireworks+1
- **Baseten** — very strong if you want both **OpenAI-compatible model APIs** and a path to **dedicated/single-tenant deployments** or even **deploying your own models**.baseten+2
- **Hugging Face Inference Endpoints** — very good if you want to deploy models from the **Hugging Face ecosystem** on **dedicated autoscaling endpoints** with less platform lock-in around model artifacts.huggingface+1
- **RunPod** — very attractive if you want to run **your own vLLM / custom containers** on GPUs and keep more control over the serving stack.runpod+1
- **Modal** — very attractive if you want a **developer-friendly serverless GPU runtime** and are comfortable defining infrastructure programmatically in Python.modal

If you want the most practical shortlist for **your** project, I would benchmark:

Together AI vs Fireworks AI vs Baseten
for **managed inference**, and
RunPod (or Modal)
for **bring-your-own serving stack**.together+4

- **How I would think about the choice**

You are not just choosing "where to run a model."
You are choosing between two PoC paths:

- **Path A — Managed inference**

You keep your app simple:

- Streamlit app
- Python backend
- OpenAI-compatible API calls

- no self-managed model server

This is easiest operationally and fastest to scale for a PoC. Together AI, Fireworks AI, Baseten Model APIs, Groq, and some Hugging Face provider flows all fit this pattern.together+4

- **Path B — Rent GPUs and run your own stack**

You rent GPU compute and run:

- vLLM
- your chosen chat model
- your chosen embedding model
- optionally your image-analysis model
- your own vector DB and application services

This is more work, but it is much closer to your eventual production architecture, especially because your final target is **fully local/private deployment**. RunPod, Lambda, Vast.ai, and Modal all support this style to different degrees.runpod+3

- **What Together AI is good at**

Together AI's current positioning is very strong for:

- **serverless inference**
- broad **model catalog**
- multiple modalities
- and deployment choices such as **serverless**, **batch**, **dedicated inference**, and **dedicated container inference**. The model catalog page and serverless docs explicitly show a wide model library and multiple deployment options.together+2

For your use case, the most relevant part is that Together AI gives you:

- **OpenAI-style API access** to many open models, and

- a path from quick serverless usage to more controlled dedicated options.together+1

So Together AI is **not** a bad choice at all.
The question is whether another provider fits your priorities better.

- **My view on the best alternatives**

- **1) Fireworks AI — best "Together-like" alternative for hosted inference**

Fireworks explicitly documents **OpenAI compatibility**, including use through the standard OpenAI Python client with a different base_url, which is directly aligned with your architectural requirement of keeping the software interface stable.fireworks

It also supports **on-demand dedicated GPU deployments**, and its deployments documentation explicitly says dedicated deployments offer:

- lower latency / higher throughput,
- no hard rate limits,
- broader model selection,
- and support for **custom models** from Hugging Face or elsewhere.mintlify
- **Why I think Fireworks is a strong candidate for you**
- It is a very natural fit if you want to keep using an **OpenAI-compatible API**.fireworks
- It gives you a path from "call an API" to "deploy my own model on dedicated GPUs."mintlify
- That makes it a nice bridge between local PoC and later production-style architecture.
- **My practical verdict**

If you like Together AI's managed-inference style, **Fireworks is probably the first alternative I would test**.fireworks+1

- **2) Baseten — best if you want enterprise-style deployment options**

Baseten's docs state that **Model APIs** provide **OpenAI-compatible endpoints** for high-performance LLMs, and that when you need a model that is not in the managed supported list — or need dedicated GPUs / custom scaling — you can deploy your own models with **Truss**.baseten

Baseten also emphasizes **dedicated deployments**, including:

- single-tenant inference,
- autoscaling,
- cloud or self-host / hybrid options,
- and secure/compliant deployment patterns.baseten
- **Why I think Baseten is very interesting for your case**
- It supports the **OpenAI-compatible interface** you want.baseten
- It has a stronger **"move to dedicated production"** story than many pure serverless providers.baseten+1
- It is a good fit if you expect the PoC to become a more serious hosted environment before the final on-prem deployment.
- **My practical verdict**

If you want something a little more **deployment-platform-oriented** than Together AI, **Baseten is one of the best alternatives**.baseten+1

- **3) Hugging Face Inference Endpoints — best if your workflow is very Hugging Face-centric**

Hugging Face documents Inference Endpoints as a **secure production solution** to deploy models on **dedicated and autoscaling infrastructure**, and the huggingface_hub docs show how to create endpoints programmatically.huggingface

They also document autoscaling, scale-to-zero, replica control, and hardware- or request-based scaling strategies.huggingface+1

- **Why this matters for your project**

Because you are likely to experiment with models that live on Hugging Face anyway, HF Endpoints can simplify:

- model artifact management,
- switching between checkpoints,
- and deployment of specific repositories.huggingface
- **The caveat**

HF Endpoints feel more like **managed model hosting** than an "all-in-one inference platform with broad provider abstraction." So if you want the most polished **OpenAI-compatible hosted inference UX**, I would still compare it against Fireworks and Baseten rather than assume it wins by default. This is my judgment based on platform focus.

- **My practical verdict**

If you expect to live heavily in the Hugging Face ecosystem, **HF Endpoints is absolutely worth evaluating**.huggingface+1

- **4) RunPod — best if you want to run your own stack on rented GPUs**

RunPod's docs describe:

- **Serverless**
- **Pods**
- public endpoints
- and guides for deploying **vLLM for text generation**.runpod+1

RunPod Serverless is explicitly positioned as:

- pay-as-you-go compute,
- GPU-powered,
- automatically scaling from zero,

- and suitable for inference-focused AI/ML workloads.runpod+1
- **Why I think RunPod is very relevant to you**

Your long-term architecture already points toward:

- self-hosted open models,
- likely vLLM,
- your own retrieval layer,
- your own app stack.

RunPod lets you get much closer to that architecture than a pure managed model API.runpod+1

- **My practical verdict**

If your goal is:

"Move the PoC to remote GPUs, but keep the serving topology similar to what we'll later run on-prem,"
then **RunPod is one of the best options to test**.runpod+1

- **5) Modal — best if you want serverless GPUs but prefer Python-defined infrastructure**

Modal documents itself as a **serverless cloud** for compute-intensive apps and explicitly highlights examples such as:

- **deploy an OpenAI-compatible LLM service**,
- batch LLM workflows,
- RAG examples,
- OCR job queues,
- and large-model serving.modal
- **Why Modal might be attractive**
- Very friendly for Python-heavy engineering teams.

- Good if you like “infrastructure as Python code.”
- Good for building PoC services that include more than just one model endpoint.
- **The caveat**

Modal is more of a **compute application platform** than a focused hosted-inference marketplace. So it is excellent if you want flexibility, but less turnkey if what you really want is “give me an OpenAI-compatible endpoint for model X.”

- **My practical verdict**

If you enjoy owning more of the service logic but don’t want raw cloud ops, **Modal is a strong alternative to RunPod**.modal

---

- **Providers I would consider but probably not prioritize first**
- **Groq**

Groq is very compelling if **speed** is your main priority. Groq documents **OpenAI compatibility** and very high token speeds, and their model page lists strong throughput numbers for supported models.groq+1

However, Groq is more specialized around the models they host and their own inference hardware. It is excellent for fast hosted inference, but if your goal is to stay close to a **custom self-host/open-weight deployment path**, I would test Fireworks/Baseten before Groq.

- **Replicate**

Replicate is excellent for:

- fast experimentation,
- lots of public models,
- custom model packaging with Cog,
- and autoscaled deployments. Their docs explicitly cover custom model deployment and scaling.replicate+1

But for a **RAG chatbot stack** where you care a lot about stable OpenAI-style interfaces and later migration toward your own serving layer, I see Replicate as more of a rapid prototyping platform than my first choice.

- **Lambda / Vast.ai**

These are very relevant if you want raw GPU compute:

- Lambda offers on-demand GPU instances and preinstalled ML stacks.lambda+1
- Vast.ai offers a GPU marketplace, serverless, and cluster options with transparent pricing.vast+1

These are strong if cost optimization and GPU access matter most, but they are farther from a turnkey managed inference API experience.

- **What I would choose for your project**

Because your plan is:

1. run local PoC,
2. move to stronger remote infrastructure,
3. keep the software interface stable,
4. later move to confidential local/private production,

I would choose based on **how close you want the remote phase to be to production**.

- **Option A — easiest remote PoC**

Choose a **managed inference provider**.

- **My shortlist**
- **Fireworks AI**
- **Baseten**
- **Together AI**

- **Why**

All three are good fits for:

- open-weight model access,
- hosted inference,
- and OpenAI-compatible integration patterns.fireworks+2
- **My ranking**
- **Fireworks AI** if you want a strong mix of hosted API + dedicated deployments.fireworks+1
- **Baseten** if you want a stronger "platform + dedicated inference + custom deployment" story.baseten+1
- **Together AI** if breadth of model catalog and easy serverless experimentation are top priority.together+2

- **Option B — best architectural continuity**

Rent GPUs and run **your own vLLM stack**.

- **My shortlist**
- **RunPod**
- **Modal**
- **Lambda**
- **Vast.ai**
- **Why**

This path lets you keep the remote phase architecturally close to your final on-prem system:

- same model server,
- same OpenAI-compatible endpoint,
- same application code,

- same retrieval stack.runpod+3
- **My ranking**
- **RunPod** first, if you want straightforward GPU deployment for vLLM.runpod+1
- **Modal** if you prefer Python-defined infrastructure and serverless orchestration.modal
- **Lambda** if you want classic GPU instances with a polished ML stack.lambda+1
- **Vast.ai** if minimizing cost is very important and you are comfortable with a marketplace model.vast+1

- **My personal recommendation**

If I were advising you pragmatically, I would say:

- **Best managed-inference alternative to Together AI**

Fireworks AI.
It is the alternative I would test first because:

- OpenAI compatibility is explicit and well-documented,fireworks
- dedicated deployments are clearly supported, including custom models,mintlify
- and it feels like a natural fit for a model-serving-centric architecture.
- **Best "closer to production" alternative**

RunPod.
If you already expect to serve your own models via vLLM, RunPod is a very good bridge between:

- your local workstation PoC,
- a more powerful remote PoC,
- and your final on-prem production stack.runpod+1

- **Best enterprise-style deployment platform**

Baseten.
If you want a bit more control, dedicated deployments, and a more platform-oriented offering, Baseten is excellent.baseten+1

- **One important caveat**

Since you ultimately want the **production system to be fully local/private** and to operate on **confidential documentation**, I would use any public-cloud provider only for:

- **non-confidential PoC data**
- and **benchmarking / model selection**.

That is consistent with the direction you already described, and it keeps the cloud phase useful without locking you into a cloud-only architecture.

If you want, I can do one of these next:

1. **compare Together AI vs Fireworks AI vs Baseten specifically for your PoC**,
2. **compare managed inference vs RunPod/vLLM for your roadmap**, or
3. **propose the best remote architecture for the next phase of your PoC**.

Sources

- **Friday**

---

11 Dual language support

You said:

I forgot to mention explicitly that also the contents of the wiki website may be in English or in Italian. Is it necessary to change the design to cope with that?

Copilot said:

Copilot

No — you do **not** need to redesign the whole system.
But you **should make a few explicit multilingual choices now**, so that the PoC works well when both the **documents** and the **user queries** may be in **English or Italian**.ollama+3

In other words:

No major architectural rewrite is needed.
Yes, a few parts of the design should become explicitly multilingual.ollama+3

- **Short answer**
- **What stays the same**

The overall RAG pipeline is still the same:

- ingest wiki pages,
- chunk content,
- create embeddings,
- store in vector DB,
- retrieve relevant chunks,
- generate an answer with citations.
  That workflow does not fundamentally depend on the content being only English.ibm+1
- **What should change**

You should make sure that:

1. the **chat model** is multilingual,
2. the **embedding model** is multilingual,
3. each chunk stores a **language field**,
4. the ingestion pipeline can **detect page/chunk language**, and
5. retrieval is designed so English queries can still find Italian chunks (and vice versa), if you want cross-language search.ollama+3

- **Why this is not a major redesign**

Modern open models already support multilingual usage:

- **Qwen3** explicitly supports **100+ languages and dialects** and is designed for multilingual instruction following and translation.ollama+1
- **Granite 4.0** explicitly supports **Italian** along with English and several other languages.ollama+1
- **Granite Embedding 278M Multilingual** explicitly supports **Italian** and English.ibm+1
- **BGE-M3** supports **100+ languages**, long documents up to **8192 tokens**, and multilingual retrieval.huggingface+1

Because of that, you do **not** need a separate English pipeline and Italian pipeline. You can still build **one RAG system**, as long as the models are chosen appropriately.ollama+3

- **What I recommend changing in the design**
- **1) Make the embedding model explicitly multilingual**

This is the most important point.

If the wiki content can be in **English or Italian**, and the user can ask in **English or Italian**, the embedding model should be able to represent both languages in a

shared semantic space. That is exactly what multilingual embedding models are for.ibm+1

- **Good options**
- **Granite Embedding 278M Multilingual** if you want a lighter model and explicit Italian support. IBM states that it supports **English, German, Spanish, French, Japanese, Portuguese, Arabic, Czech, Italian, Korean, Dutch, and Chinese**.ibm+1
- **BGE-M3** if you want a stronger retrieval-oriented model with support for **100+ languages**, long inputs, and future hybrid retrieval options.huggingface+1
- **My recommendation**

For your project, the design does **not** need to change structurally, but I would make sure the embedding layer is one of these multilingual models from the beginning.ibm+1

- **2) Add a language field to every page and chunk**

Even if you use multilingual embeddings, I still strongly recommend storing language metadata such as:

JSON

Why?

- it helps with debugging,
- it enables language-based filtering,
- it helps evaluation,
- and later it lets you decide whether answers should prefer chunks in the same language as the query.

This recommendation is architectural rather than something the docs state directly, but it follows naturally from the multilingual capabilities of the embedding and generation models.ibm+3

- **Practical rule**
- detect language at **page level**
- and optionally at **chunk level** if pages may contain mixed-language sections

For a technical wiki, page-level detection is usually enough for v1.

- **3) Keep chunking the same, but avoid mixing languages in one chunk when possible**

You do **not** need a new chunking algorithm.
But if a page contains both English and Italian blocks, avoid creating mixed-language chunks unless the original structure forces it.

- **Good practice**

Chunk by:

- heading/section,
- paragraphs,
- tables,
- image summaries

and preserve the original language of each segment.

That gives the multilingual embedding model cleaner input and improves retrieval consistency. This is design guidance, but it aligns well with the fact that your embedding choices are designed for multilingual retrieval rather than noisy mixed-language fragments.ibm+1

- **4) Make the chat model multilingual too**

The conversational model should be able to:

- understand English and Italian questions,
- answer in the user's language,
- and summarize retrieved text without degrading badly on Italian.

- **Good options**
- **Qwen3**: supports **100+ languages and dialects** with strong multilingual instruction following and translation.ollama+1
- **Granite 4.0**: explicitly supports **Italian** and is intended for multilingual dialogue and RAG use cases.ollama+1
- **My recommendation**

This does **not** require redesigning the system, but it does mean I would avoid a chat model that is primarily English-only.
For your current shortlist, both **Qwen3** and **Granite 4.0** are compatible with this requirement.ollama+1

- **5) Decide whether you want same-language retrieval or cross-language retrieval**

This is the one design choice you should make explicitly.

- **Option A — same-language preference**

If the user asks in Italian, prefer Italian chunks first.
If the user asks in English, prefer English chunks first.

This often feels more natural to users and gives more readable citations.

- **Option B — true cross-language retrieval**

Allow an English query to retrieve Italian chunks and vice versa.

This is often useful in multilingual documentation because the "best answer" may exist only in one language. A multilingual embedding model such as Granite Embedding Multilingual or BGE-M3 is exactly what enables this.ibm+1

- **My recommendation**

For your project, I would design for:

- **cross-language retrieval enabled**
- but with a **same-language preference bias** in ranking or filtering

That gives you the best of both worlds.

- **What about image analysis?**

You do **not** need to redesign the image-analysis stage either, but you should be aware of one nuance:

- If the image model only works best with English prompts, that is still okay for ingestion.
- You can analyze images in a fixed internal prompt language and store the extracted facts as text plus metadata.
- The retrieved facts can then be summarized to the user in Italian or English by the chat model.

So multilingual wiki content does **not** force a new image pipeline architecture.
It mainly affects:

- the language of nearby text,
- metadata,
- and how the final answer is produced.

- **What I would change concretely in your current plan**
- **Minimal changes**

Here is the smallest set of design updates I would make:

- **Ingestion**
- detect and store page_language
- optionally store chunk_language
- **Metadata**

Add fields like:

JSON

- **Embeddings**

Use a **multilingual embedding model**

- **Granite Embedding 278M Multilingual** or
- **BGE-M3**ibm+1
- **Generation**

Use a **multilingual chat model**

- **Qwen3**
- or **Granite 4.0**ollama+1
- **Retrieval policy**

Implement:

- same-language preference if possible,
- fallback to cross-language matches if needed

- **My bottom line**

No, you do not need to change the overall design.
Yes, you should make the system explicitly multilingual at the model-selection and metadata levels.ollama+3

If I reduce it to one practical sentence:

Keep one RAG architecture, but choose multilingual models and store language metadata per page/chunk.ollama+3

- **My concrete recommendation for your PoC**

For your first version, I would now recommend:

- **Chat model: Qwen3** or **Granite 4.0** because both support multilingual use, and Granite explicitly includes Italian.ollama+1
- **Embedding model: Granite Embedding 278M Multilingual** if you want lightweight EN/IT support, or **BGE-M3** if you want broader multilingual and stronger retrieval flexibility.ibm+1

- **Schema update:** add page_language and chunk_language
- **Retrieval update:** prefer same-language results but allow cross-language fallback

---

If you want, I can do the next practical step and give you either:

1. **an updated metadata/schema design for multilingual EN/IT wiki ingestion**, or
2. **a revised model recommendation matrix now that the corpus is bilingual**.

Sources

- **Sunday**

---

12 Supporting other types of documents

You said:

If the system performs well once implemented, I might consider using it for other use cases too. Assuming these use cases' knowledge base consists of documents and sources other than a Mediawiki website, which parts of the design have to be modified?

Copilot said:

Copilot

Yes — **the design can be reused for many other knowledge-base types**, and that is actually one of the strongest reasons to build the PoC in a modular way. The core RAG pipeline does **not** depend on MediaWiki specifically; what changes is mainly the **ingestion and normalization layer**. Tools such as **Apache Tika**, **Docling**, and **Unstructured** exist precisely because many different document/file types can be converted into a more uniform representation before indexing. Tika detects and extracts text/metadata from **over a thousand file types** through a single interface, while Docling and Unstructured both support a broad range of enterprise document formats such as **PDF, DOCX, PPTX, HTML, images, and more**.apache+4

So the clean answer is:

You do not need to redesign the whole system.
You need to replace or generalize the source-specific ingestion components.apache+2

- **What stays the same**

The following parts of the design should remain essentially unchanged across different knowledge bases:

- **1) Retrieval and answer-generation core**

The logic:

- chunk content,

- embed chunks,
- store them in a vector store,
- retrieve the best matches,
- and generate an answer with citations,

is still the same whether the source is a MediaWiki page, a PDF, a DOCX manual, or a folder of HTML files. This is exactly why unified extraction frameworks are useful: they normalize many input types into a common text-and-structure representation that downstream AI/RAG systems can consume.apache+3

- **2) Vector database and retrieval strategy**

Your vector DB choice (for example Qdrant/Weaviate or similar), multilingual embeddings, metadata filtering, and RAG answer-generation loop do not fundamentally depend on the original source format. What matters is that the upstream pipeline produces consistent chunks and metadata. The fact that Docling explicitly exports structured representations suitable for **AI, RAG, and agentic systems** is a good example of how the downstream layer can stay stable even while document types vary.docling+2

- **3) Chat model layer**

The conversational model can remain the same, assuming it is already multilingual enough for your corpus and users. If your current chat model works well for English and Italian technical QA, then changing from MediaWiki to PDFs or Word files does **not** require a different chat architecture by itself. The main issue is making sure the ingestion layer still produces high-quality chunks and citations. This follows from the separation of concerns between extraction and downstream QA seen in tools like Docling and Unstructured, which are specifically designed to feed LLM workflows.github+1

- **What has to be modified**

The components that need to change are the ones that currently assume the source is a MediaWiki website.

- **1) The source connector / ingestion adapter**

This is the **main component that changes**.

Today your source adapter is something like:

- static HTML mirror parser,
- or later MediaWiki API connector.

For other knowledge bases, you need a different adapter depending on the source:

- filesystem folder of PDFs,
- SharePoint / OneDrive exports,
- S3 bucket,
- local DOCX/PPTX/XLSX files,
- HTML documentation site,
- e-mail archives,
- XML exports,
- scanned images, etc.

This is where tools like **Apache Tika**, **Docling**, and **Unstructured** become useful:

- **Tika** gives a single interface for detecting and extracting text/metadata from a very large variety of file types.apache+1
- **Docling** supports PDF, DOCX, PPTX, XLSX, HTML, images, audio, Markdown, CSV, LaTeX, and more, and converts them into a unified DoclingDocument structure.github+2
- **Unstructured** supports many file types including PDF, HTML, Word, PowerPoint, text, email, XML, spreadsheets, and images, and is explicitly built for preprocessing data for downstream LLM workflows.unstructured+2
- **Practical design implication**

Instead of one hard-coded "MediaWiki ingestor," define an interface such as:

Plain Text

Then implement adapters like:

- MediaWikiApiAdapter
- StaticHtmlAdapter
- PdfFolderAdapter
- SharePointAdapter
- DoclingAdapter
- TikaAdapter

This is the single most important design generalization.

- **2) The document parsing and normalization layer**

Once you leave MediaWiki, raw extraction becomes much more heterogeneous. HTML pages, PDFs, DOCX files, slide decks, spreadsheets, and scanned pages do **not** expose the same structure.

So you need a stronger **normalization layer** that converts each source into a canonical internal format such as:

JSON

This is where Docling is especially interesting, because it explicitly provides a **unified, expressive document representation** and exports to formats such as Markdown, HTML, JSON, and DocTags, while preserving layout, reading order, tables, formulas, images, and more.github+2

Apache Tika is also strong here, though more focused on content/metadata extraction than on deeply preserving layout semantics. Tika is valuable when you need a broad, single extraction interface across many enterprise file types.apache+1

- **Practical design implication**

You should standardize around an **internal canonical document model**.
That is the key abstraction that lets everything downstream remain stable.

- **3) The chunking strategy**

The chunking layer stays conceptually the same, but the **rules** will need to adapt to the type of source material.

- **For MediaWiki / HTML**

Chunking usually follows:

- page title,
- headings,
- paragraphs,
- lists,
- tables,
- image captions.

- **For PDFs / DOCX / PPTX / reports**

Chunking often has to respect:

- page layout,
- reading order,
- section hierarchy,
- table boundaries,
- slide boundaries,
- captions,
- footnotes,
- headers/footers.

Docling explicitly highlights support for:

- reading order,
- table structure,
- bounding boxes,
- code,

- formulas,
- captions,
- image classification,
- and chunking for AI ingestion.docling+1

Unstructured likewise documents format-specific processors for PDF, DOCX, PPTX, HTML, email, text, and more, all of which convert content into structured element objects that can then be chunked.github+1

- **Practical design implication**

You do **not** need a new chunking subsystem every time, but you should make chunking **document-type-aware**:

- chunk_html()
- chunk_pdf()
- chunk_docx()
- chunk_slide_deck()
- chunk_spreadsheet()

or, better, chunk the **canonical normalized representation** in different ways depending on the source type.

- **4) The OCR / image / table enrichment stage**

This layer becomes **more important**, not less, when the source is not a website.

With MediaWiki, much of the content is already machine-readable HTML.
With other knowledge bases, you may face:

- scanned PDFs,
- screenshots,
- diagrams inside PPTX,
- embedded tables,

- photographed documents,
- invoices/forms.

Docling explicitly supports:

- OCR for scanned PDFs and images,
- image classification,
- table structure extraction,
- advanced PDF understanding,
- and even audio formats and structured exports.github+2

Unstructured also supports a wide set of text/image/document formats and is designed to convert them into structured data for LLM workflows.unstructured+2

- **Practical design implication**

If the new use cases involve many **scanned** or **image-heavy** documents, you will need to strengthen:

- OCR,
- table understanding,
- chart understanding,
- image-caption generation,
- and attachment enrichment.

So this part of the design is not replaced, but it often becomes more central.

- **5) The metadata schema**

This definitely needs to become more general.

For MediaWiki, relevant metadata may include:

- page title,
- namespace/category,
- section path,

- page URL,
- revision ID.

For other sources, you may need:

- file path,
- file type,
- document author,
- creation/modification date,
- storage location,
- page number,
- slide number,
- attachment parent,
- source system,
- access-control tags,
- business unit / product tags.

Tika explicitly emphasizes metadata extraction across many file types, which is a strong reminder that metadata varies greatly by source format and should be treated as a first-class layer.apache+1

- **Practical design implication**

Generalize your metadata schema to include:

- **universal fields** (doc_id, source_type, title, language, chunk_id),
- plus **source-specific fields** under a nested object.

Example:

JSON

- **6) The change-detection / synchronization logic**

If the source is no longer a MediaWiki website, then your synchronization method will also change.

For MediaWiki production, you already discussed:

- recent changes,
- hooks,
- revision-aware updates.

For other sources, updates might come from:

- filesystem watchers,
- cloud storage events,
- document management system webhooks,
- scheduled re-scan jobs,
- content hashes,
- connector-specific sync metadata.

This means the **synchronization adapter** should be generalized too:

Plain Text

Example implementations:

- MediaWikiChangeDetector
- FilesystemChangeDetector
- S3ChangeDetector
- SharePointChangeDetector

The downstream “replace old chunks with new chunks” logic can stay the same; only the way changes are discovered needs to vary.

- **A useful rule of thumb**

The design should be split into two broad layers:

- **Source-specific layer**

This is what changes per use case:

- connectors
- raw parsers
- OCR strategy
- metadata extraction
- sync/change detection
- **Source-agnostic layer**

This should stay stable:

- canonical document model
- chunk storage format
- embedding
- vector DB
- retrieval
- re-ranking
- answer generation
- UI/API

That is the architecture I would explicitly aim for.

- **Recommended generalized architecture**

I would evolve your current design into this:

Plain Text

This is the cleanest way to ensure the system can later serve other knowledge bases with limited redesign.

- **My direct answer**

- **Which parts have to be modified?**

The parts that need to be modified for non-MediaWiki knowledge bases are mainly:

1. **source connector / loader**apache+2
2. **parsing and normalization logic**github+2
3. **chunking rules for different document structures**docling+1
4. **OCR / image / table enrichment**, especially for scanned or visual documentsgithub+2
5. **metadata schema**, to support source-specific metadata beyond MediaWiki fieldsapache+1
6. **change-detection / synchronization adapter**, because each source has its own update mechanism

- **Which parts should remain the same?**
- vector DB
- embedding strategy
- retrieval and RAG logic
- answer generation
- UI/API
  These should remain largely stable if you introduce a **canonical internal document model**.github+2

- **My recommendation**

If you think this system may later support multiple knowledge-base types, I would explicitly make **two design decisions now**:

- **1) Build a canonical document schema now**

That is the key enabler for reuse.

- **2) Build the ingestion pipeline as a plug-in/adapter architecture**

Do **not** hardcode MediaWiki assumptions into the rest of the system.

If you do that, moving from:

- MediaWiki to
- PDFs / DOCX / PowerPoint / SharePoint / file shares / HTML docs

becomes an **ingestion extension project**, not a full redesign.

If you want, I can next do one of these:

1. **design a source-agnostic canonical document schema for your RAG system**, or
2. **propose a modular architecture showing exactly which Python modules should be MediaWiki-specific and which should stay generic**.

Sources

- **Yesterday**

---

13 Defining APIs

You said:

In order to make the system usable for possible, different use cases, I would like to define an API for the chat backend and one for the ingestion pipeline.The first should be used for sending questions and receiving responses. The second should be used primarily for injecting new documents/data to the propretary knowledge base.Are there standardized API for these operations or do you think it would be better to create from scratch new APIs?

Copilot said:

Copilot

You're thinking in exactly the right direction: **define stable APIs for (1) chat interaction and (2) ingestion**, so that your system becomes *extensible and reusable* across many different knowledge bases and deployments.

And the good news is:

There is no single "industry standard" API for LLM-RAG ingestion or RAG chat, but several de-facto patterns exist.
You can **reuse these patterns** so you don't create something strange or proprietary, while still keeping full flexibility.

Let's break it down.

- **1. API for the chat backend**

This is the easy one — you *should not* invent anything new.

Use a **fully OpenAI-compatible API**, because:

- It is now the **de-facto standard** for LLM/Chat workloads everywhere.
- It is what **vLLM**, **Ollama**, **Fireworks**, **Baseten**, **Together AI**, etc. already support.
- It is the simplest way to "swap" models or hosting environments without changing your business logic, UI, or retrieval layer.
- **What the chat API should expose**

You should expose **one endpoint** mirroring POST /v1/chat/completions, e.g.:

POST /v1/chat/completions

{

"model": "my-chat-model",

"messages": [...],

"stream": false

}

Why?

- Your Streamlit app already integrates nicely with this shape.
- Model serving can move from local → cloud → local again without changing anything.
- The RAG backend can treat your chat API *as if it were OpenAI*.

In other words: the chat interface *is already standardized.*
Use it. Do not invent another protocol.

- **2. API for the ingestion pipeline**

This is where things get tricky.

There is *no industry-wide standard ingestion API* for RAG systems.
However, there **is** a strong emerging ecosystem of *standardized document ingestion frameworks* (Docling, Unstructured, Tika) — but **no shared HTTP API spec** for feeding them into vector databases or RAG engines.

Because of this, teams typically design a **custom ingestion API**, but based on common patterns.

- **2.1. What ingestion API patterns exist "in the wild"?**
- **Common patterns used by real RAG platforms**

Platforms like LlamaIndex, LangChain, Weaviate, Qdrant, and enterprise RAG products tend to support:

1. **Upload documents**
2. **Delete documents**
3. **Update documents**
4. **Retrieve document metadata**
5. **Trigger re-indexing**
6. **Check ingestion job status**

- **Why no standard?**

Because ingestion depends on:

- document types (PDF, DOCX, Wiki, HTML, CSV, images…)
- chunking policy,
- metadata,
- custom preprocessing,
- OCR,
- image analysis,
- security classification.

These vary so much that no universal HTTP ingestion API has emerged.

- **2.2. What you should do**

You should create **your own ingestion API**, but in a way that is:

- **generic** (works for any knowledge base),
- **stable** (doesn't change per use case),
- **modular** (plug in new connectors),
- **extensible** (metadata, chunking strategies),

- **Asynchronous** (heavy work happens in background jobs).

The ingestion API should NOT depend on MediaWiki at all.
It should operate at a higher layer, above document source types.

- **2.3. Recommended ingestion REST API**

Here is the minimal, future-proof API structure I recommend.

- **(A) Upload a new document**

POST /v1/ingestion/documents

Content-Type: multipart/form-data

file: <uploaded document>

metadata: { ... }

Why multipart? Because documents may include:

- PDFs
- ZIP archives
- HTML bundles
- MediaWiki XML exports
- images
- spreadsheets
- Office files

- **(B) Ingest by reference (URL, file path, API source)**

POST /v1/ingestion/sources

{

"source_type": "mediawiki_api" | "s3" | "filesystem" | "sharepoint" | ...

"config": { ... }

}

This supports:

- MediaWiki ingestion,
- Document repositories,
- File servers,
- Cloud storage,
- CRMs / ticketing systems etc.

- **(C) Delete a document**

DELETE /v1/ingestion/documents/{doc_id}

This removes:

- chunks,
- embeddings,
- metadata,
- stored files.

- **(D) Update (replace) a document**

PUT /v1/ingestion/documents/{doc_id}

multipart/form-data

or

POST /v1/ingestion/documents/{doc_id}/replace

This is important for synchronizing dynamic sources like wikis.

- **(E) Trigger batch ingestion jobs**

POST /v1/ingestion/jobs

{

"job_type": "reindex_all" | "sync" | "verify" | "purge"

}

- **(F) Check job status**

GET /v1/ingestion/jobs/{job_id}

- **(G) Retrieve document metadata**

GET /v1/ingestion/documents/{doc_id}

- **2.4. Why this API structure is future-proof**

Because:

- It does not assume **MediaWiki** specifically.
- It does not assume a specific **document format**.
- It does not assume a specific **chunking** or **embedding** strategy.
- It acts as a generic interface to **any ingestion backend** — whether your backend uses:
    - Apache Tika (mass format coverage),
    - Docling (deep structured parsing for PDFs/DOCX),
    - Unstructured (enterprise connectors),
    - or your own code.

Using this API, you can plug in:

- PDFs → via Tika or Docling

- DOCX → via Docling or Unstructured
- Scanned documents → via OCR pipeline
- HTML sites → via a static-site parser
- Wikis → via MediaWiki API
- JSON/CSV → via custom importer
- Versioned file stores → via event-based detectors

All of these feed the same ingestion API.

- **3. Should you adopt an external standard anyway?**

There is **no standard ingestion API** like OpenAI Chat Completions.
But you can adopt conventions from:

- **OpenAI-style REST patterns**
- **JSON:API**
- **REST-ish resource design**
- **Async job patterns (status polling)**

For example:

- asynchronous job creation → 202 “job accepted”
- job status endpoint → GET /v1/jobs/<id>
- document resource endpoints → CRUD semantics

This makes your API intuitive and maintainable.

- **4. My final recommendation**
- **Use a standardized API for chat:**
- **OpenAI-compatible**
- Never invent your own chat interface

- All model hosts and serving stacks already support it
- **Build a custom ingestion API, but using industry-common patterns:**
- document upload
- document replace/delete
- ingestion job creation
- job status
- metadata endpoints

This gives you maximum:

- flexibility,
- future reuse,
- easy extension to new data sources,
- separation between ingestion and retrieval,
- alignment with your long-term plan to support many knowledge bases.

- **If you want, I can draft:**
- the **complete API specification** (OpenAPI 3.1)
- Python FastAPI templates for both APIs
- a plug-in ingestion architecture
- design patterns for asynchronous indexing jobs

Just tell me what you'd like next.

- **Today**

---

14 Adding agentic functionalities

You said:

In the future, it might be interesting to add agentic capabilities to the system. For example, users could ask to summarize the information included in the knowledge base related to a certain topic, producing a PDF document.Does this affect the design?

Copilot said:

Copilot

Short answer: **Yes — but only in small, predictable ways.**
You do *not* need to redesign the system from scratch.
You *do* need to add **one more architectural layer** that makes agentic workflows first-class citizens.

Your current design (chat API + ingestion API + RAG backend) is an **excellent foundation**, and the additions required for agents are evolutionary, not disruptive.

Below is the clear, practical breakdown.

- ✅ **1. Why agentic capabilities matter**

Examples like:

- *"Summarize everything the KB says about X."*
- *"Produce a PDF report combining these topics."*
- *"Find all documents issued in 2021 and compare them."*
- *"Generate a table with key metrics from the KB."*

…are **agent tasks**, not simple single-turn question answering.

This means the backend needs the ability to:

- **run workflows**,
- **perform multi-step reasoning**,
- **call tools**,

- **write files**,
- **invoke ingestion/read APIs**,
- **assemble intermediate results**,
- **produce final artifacts** (e.g., PDF, DOCX, CSV).

But none of that breaks your current design if you plan for agents now.

- ✅ **2. The key observation**

Agent capabilities **do not change**:

- Your ingestion API
- Your vector store
- Your embedding strategy
- Your chunking pipeline
- Your document model
- Your chat API interface (OpenAI-compatible)

All of these remain future-proof.

The *only* part that needs extension is:

a new "Agent Orchestration Layer" between the chat API and backend tools.

- ✅ **3. What gets added: an Agent API (and internal tool layer)**
- **You keep:**
- **/v1/chat/completions** (OpenAI-style chat)
- **/v1/ingestion/...** (upload, replace, delete)
- **And you add a *third* surface:**
- **/v1/agent/** (for long, tool-using, multi-step jobs)

This is exactly how:

- OpenAI "Assistants" API,
- LangGraph,
- LlamaIndex Agents,
- CrewAI,
- OpenAI's "Responses API",
- and Anthropic "Tool use"

structure things.

- **Why a separate API?**

Because agent tasks are:

- **long-running**
- **stateful**
- **asynchronous**
- **tool-calling**
- **may produce files (PDF, CSV, ZIP)**

This is very different from the stateless chat endpoint.

- ✅ **4. Minimal Agent API you would need**

Here is a lean, future-proof structure:

- **(A) Submit an agent task**

POST /v1/agent/tasks

{

"instructions": "Summarize all info about lithium batteries and generate a PDF",

"parameters": {...},

"stream": false

}

Returns:

JSON

- **(B) Check task status**

GET /v1/agent/tasks/{task_id}

- **(C) Retrieve results (files, text, artifacts)**

GET /v1/agent/tasks/{task_id}/result

- **(D) Stream progress (optional)**

GET /v1/agent/tasks/{task_id}/events

This maps perfectly to long operations like:

- scanning the KB
- retrieving dozens of chunks
- performing reasoning
- generating a summary
- creating a PDF

- ✅ **5. Internal architecture addition: the Tool Layer**

Agents must call tools such as:

- **SearchTool** → query vector DB
- **DocumentTool** → read raw documents
- **WriteFileTool** → generate PDFs
- **IngestionTool** → fetch metadata, list docs
- **Math / Graph / TableTools**
- **HTTPTool** → call 3rd-party APIs
- **FilesystemTool** → temporary storage

This is **not exposed** directly to clients.
It lives *inside* the backend.

A typical internal flow:

(Chat message)

→ Agent Runtime

→ Planner (LLM)

→ Tool call 1 (search KB)

→ Tool call 2 (aggregate data)

→ Tool call 3 (generate PDF)

→ Store artifact

→ Return result

This is precisely how agent frameworks like LangChain and LangGraph work.

- ✅ **6. Does this affect your ingestion API?**

Not materially.

Your ingestion API remains:

- stateless,
- document-oriented,
- synchronous or async,
- safe,
- predictable.

Agents may **call the ingestion API internally** (e.g., "add this document to the KB"), but that doesn't change its design.

If anything, it reinforces your decision to separate ingestion from chat.

- ✅ **7. Does this affect your chat API?**

No.

That stays identical:

- OpenAI-compatible
- Stateless
- Suitable for your Streamlit UI

You do **not** want agent and chat to share the same API surface.

- ✅ **8. Does this affect your vector store / embeddings?**

No.

Agents simply use the retrieval layer multiple times.

- ✅ **9. Does this require changing the document model?**

No — but you should ensure the model includes:

- document titles
- section markers
- timestamps
- source URI
- tags or classification metadata

These are useful for agents performing:

- topic clustering
- summarization
- timeline analysis
- multi-document comparisons

But the schema itself does not fundamentally need redesign.

- 🧩 **10. What you *should* add now (to stay future-proof)**

If agentic capabilities are in your long-term plan, the following early design decisions help:

- **1. A job/task infrastructure**

Agents need asynchronous job execution.
Add:

- job table / queue
- task statuses
- worker processes

- **2. Ability to generate artifacts**

Ensure your backend can create:

- PDF
- DOCX
- CSV
- ZIP

This is easy with Python (ReportLab, WeasyPrint, Pandoc wrappers).

- **3. Tool execution boundaries**

Define a plugin-style architecture:

/tools

search.py

summarize.py

pdf_generator.py

ingestion_client.py

scrape.py

...

- **4. Temporary working storage**

Agents need access to:

- temporary directories,
- ephemeral storage,
- cleanup routines.

- 🎯 **Final answer**

Yes, adding agentic capabilities affects the design — but only in one place:
you need to add an Agent Orchestration Layer and a new Agent API.

Everything else in your system (chat API, ingestion API, retrieval backend, vector DB, embeddings, document model) remains valid and future-proof.

Your final architecture would look like:

```
        [CHAT API]  ← OpenAI-compatible
            |
  [Agent API for multi-step tasks]
            |
      [Agent Runtime]
   ↙        ↓        ↘
[Retrieval] [Tools] [Document Loader]
            |
        [Vector DB]
            |
       [Ingestion API]
```

This is the same architectural pattern used by modern agentic systems like:

- OpenAI Assistants
- LangGraph

- LlamaIndex Agents
- CrewAI

If you want, I can now help you design:

- ✔ the **exact API specification** for the Agent API (OpenAPI 3.1)
- ✔ the internal tool-calling architecture
- ✔ or a complete reference diagram for your end-to-end multi-agent system

Just tell me what you'd like next!

15 Users' roles

Would it be possible to implement different roles for the users so that they can access different portions of the knowledge base?

Copilot said:

Copilot

Yes — **it is absolutely possible** to implement user roles and restrict access to different parts of the knowledge base.
Even better, this can be added **incrementally** without redesigning the core RAG architecture.

Below is a clear breakdown of how to support *role-based access control (RBAC)* or even *attribute-based access control (ABAC)* in a retrieval-augmented system.

✅ What "restricted access" means in a RAG system

When you say users may access "different portions of the knowledge base," this usually maps to one or more of these:

- **document-level restrictions**
  e.g., only Finance users can see finance docs
- **section-level or chunk-level restrictions**
  e.g., engineers may see internal technical diagrams, but sales cannot
- **source-level restrictions**
  e.g., one team's wiki vs another team's documents
- **tag-based restrictions**
  e.g., confidential, secret, public, customerA-only

All of these can be enforced **before retrieval**, **during retrieval**, or **after retrieval** — depending on what is safest or easiest.

🧩 Where to add role-awareness in the architecture

You do **not** need to modify the ingestion pipeline or the RAG architecture deeply. You only need to add:

1. **User authentication**
2. **Roles / permissions stored in the backend**
3. **Metadata on chunks indicating security labels**
4. **Filtered retrieval queries to the vector DB**

That's it.

Below is a complete but simple design.

🧱 Step-by-step design

1. Add user roles to the authentication layer

Even in a local PoC, this can be as simple as:

JSON

In a more advanced setting:

- connect to an OAuth provider (Azure AD, Google Workspace, etc.)
- include roles or group memberships in JWT claims

But the system only needs:

- **who the user is**
- **what roles they have**

---

2. Add metadata to each chunk during ingestion

During ingestion, each chunk should carry a field like:

JSON

You can attach:

- roles

- clearance levels
- visibility tags
- department ownership
- ABAC attributes (region, customer, product line, etc.)

This metadata is indexed together with the chunk.

---

3. Filter retrieval queries in the vector store

Most vector DBs support **metadata filtering**:

- Qdrant → payload filters
- Weaviate → where filter
- Pinecone → metadata filter
- ChromaDB → filtering
- Elasticsearch → index filters

Example (Qdrant-style pseudo-query):

```
search(
    query_vector=...,
    filter={
        "must": [
            {"key": "roles_allowed", "match": {"any": ["engineer", "internal"]}}
        ]
    }
)
```

This ensures that:

the model never even sees restricted chunks

This is the safest possible approach.

---

4. (Optional) Apply a final safety check after retrieval

Sometimes documents mix confidential and non-confidential content.
A final check can remove chunks that don't match the user's role.

But if you filter at retrieval time, this is often unnecessary.

🔐 **RBAC vs ABAC — recommended model**

RBAC (Role-based access control)

Simple. Works well if permissions are stable:

- engineer
- support
- admin
- sales
- HR

ABAC (Attribute-based access control)

More flexible:

- document sensitivity (public, internal, confidential, secret)
- team ownership
- region restrictions
- customer-specific segmentation

ABAC is ideal for enterprise knowledge bases, but RBAC is fine for your early version.

🌐 **Does this affect the ingestion pipeline?**

Only slightly:

- You must assign **visibility metadata** for each document or chunk.

- This can be done:
    - manually,
    - using folder/source rules,
    - using document tags (MediaWiki categories, SharePoint labels),
    - using a classifier later if needed.

But the pipeline itself does not need major redesign.

🤖 Does this affect the chat or RAG logic?

No, not structurally.

You simply modify the retrieval query to include:

- the user's role
- the allowed metadata

Everything else remains the same.

⭐ Example flow

1. User logs in → system knows their roles.
2. User asks a question through the UI.
3. Chat API receives the message with user_roles = ["engineering"].
4. RAG orchestrator embeds the query.
5. Vector DB search uses:
6. filter roles_allowed ∩ user_roles ≠ ∅
7. Only allowed chunks are returned.
8. Chat model generates the answer from safe context.

🏁 Final answer

Yes — you can implement user roles and restrict access to different parts of the knowledge base.
It integrates cleanly into your existing RAG architecture.

All you need is:

1. **User roles** (auth layer)
2. **Chunk metadata** (ingestion)
3. **Vector DB metadata filtering** (retrieval)
4. Optional final **post-retrieval filtering**

This gives you a system that can support:

- team-based visibility
- confidential/internal segmentation
- source-based restrictions
- future agentic workflows with security boundaries

…without changing the fundamental design.

If you'd like, I can generate:

- a **Mermaid diagram of the RBAC-enabled architecture**,
- a **metadata schema** for secure chunks,
- example code for **Qdrant / Weaviate RBAC filtering**,
- or JWT-based role propagation example for FastAPI/Flask.

16 Exploiting user-provided contextual information if any

Let's say that at some point in the future I want to add support for regular files such as PDF to build the knowledge base. If the user know some relevant information about the file (for example that it is associated with a specific topic), how to exploit this to improve/facilitate the retrieval process?

Copilot said:

Copilot

Yes — and you should definitely exploit user-provided contextual information because it dramatically improves retrieval quality.

This kind of information is called human-supplied metadata, and it is one of the most powerful—but underused—features of a RAG system.

Below is a clear guide to how you can use it, how to capture it, and how to make retrieval far more accurate and controllable.

✅ Why user-provided metadata matters

When adding PDFs (or any regular files), users often know things that the machine cannot reliably infer, such as:

“This file belongs to topic X”

“This is related to product Y”

“This is internal to the support team”

“This is about safety compliance”

“This applies only to customer ACME Corp”

“This is version 2021 of the same document”

“This is a root overview document”

“This section is especially important”

All of these are valuable signals that can:

Guide the ingestion pipeline

Strengthen metadata filtering

Improve answer accuracy

Increase recall for niche topics

Allow role-based access control

Enable topic-aware summarization

🎯 Where to use user-provided metadata

There are six strategic places where metadata boosts performance:

1️⃣ Metadata stored with each document and chunk

During ingestion you generate chunks like:

JSON

These metadata fields become filters and signals during retrieval.

2️⃣ Vector DB metadata filtering (most important)

User-provided metadata enables role filtering and topic filtering.

Example: If the user tags the PDF with topics = ["robotics", "maintenance"], your retrieval query can do:

Query embedding + metadata filter

Python

This prevents irrelevant chunks from being retrieved AND increases speed.

Benefits

precision increases massively

no more “hallucinated context”

faster queries because fewer chunks match

multi-topic repositories become manageable

3️⃣ Boosting relevance using hybrid retrieval

Modern vector DBs (Qdrant, Weaviate, Elasticsearch, Vespa) let you “boost” certain metadata fields.

Example: If a document is explicitly tagged with topic “batteries,” you can artificially increase its ranking:

score = vector_score + topic_boost * weight

This ensures the system “listens” to human context.

---

4 Reranker stage with metadata weighting

After the initial vector search, you feed results to a cross-encoder reranker:

The reranker considers both *text* and *metadata*

You can add a little bias:

“If this document matches the user’s known topic → boost its reranker score”

This gives you the best of both worlds:

vector similarity

human taxonomy guidance

---

5 Topic-specific embeddings (optional but powerful)

If the user-provided topic is known at ingestion time, you can create:

a topic embedding

store it along the chunk

use it as an auxiliary vector during search

Example: For a document tagged “hydraulics,” create:

topic_embedding = embed("hydraulics")

Then, during retrieval:

```
query_vector = combine(
    embed(user_query),
    topic_embedding,
    weights=[0.7, 0.3]
```

)

This technique dramatically improves recall in specialized domains.

6️⃣ Hierarchical / topic-aware chunking

User metadata can refine chunking:

If the user labels the PDF "related to topic X,"
→ you can add that label to every chunk.

If the PDF has sections,
→ you can map section titles to user topics.

Better chunk metadata → better retrieval.

🔧 How to collect the metadata

When a user uploads a PDF:

Ask them optional questions:

What topic is this about?

Which team uses this document?

Which product does it relate to?

What tags would you assign?

Is this public, internal, or confidential?

What year/version is this from?

Or simplest: a free-text field:
"Describe this document in 1-2 sentences."

That description can be:

stored verbatim

embedded to create a semantic anchor

used as metadata

🧠 Example flow

User uploads PDF → enters tags: ["lithium batteries", "risk assessment"].

During ingestion:

Parse / OCR the PDF

Chunk text

Add metadata to each chunk

Embed chunks

Insert into Vector DB with metadata

Done

Later, user asks:

“What are the key risks when handling Li-Ion batteries?”

Your retrieval query uses metadata filter:

topics contains "lithium batteries"

Result:

Only relevant chunks retrieved

Model produces a precise, grounded answer

No contamination from unrelated documents

🏁 Final Guidance

- ✔ Yes — user-supplied metadata is extremely valuable
- ✔ You should store it in chunk metadata
- ✔ You should use it in metadata filters
- ✔ You should use it in reranker weighting
- ✔ You can embed it to enhance semantic retrieval

Doing this will give you:

higher precision

better relevance

faster queries

more explainable answers

easier scalability as KB grows

And it is completely compatible with your existing ingestion and RAG design.

If you want, I can also generate:

- 📌 A metadata schema for chunk-level tagging (JSON)
- 📌 A diagram showing metadata flow through ingestion and retrieval
- 📌 Example code for Qdrant/Weaviate metadata filtering
- 📌 A UI mockup for PDF upload + metadata form

Just tell me!

17 Connection to MCP servers

If in a future version I would like to add agentic capabilities requiring the connection to MCP servers, how would the current design be affected?

Copilot said:

Copilot

Below is a clear, architectural answer describing how your current Q&A-only + (future) agentic architecture would be affected if you later add MCP support.

This answer is conceptual, vendor-agnostic, and directly actionable for your design.

✅ Short answer

Your current design remains valid.
To add MCP (Model Context Protocol) agentic capabilities in the future, you only need to introduce:

an Agent Orchestration Layer (which you already planned), and

an MCP Client / MCP Bridge inside this layer.

Everything else — ingestion, RAG engine, storage, chat API — stays unchanged.

In other words:

MCP support does not break your existing architecture.
You only need to add a new module, not redesign the system.

🧠 First: What MCP actually is

MCP (Model Context Protocol) is a standard for connecting LLMs to external tools, data sources, and capabilities in a secure, structured way.
It defines how:

agents discover tools

agents call tools

tools return results

capabilities are described

prompts and responses flow

You will act as an MCP client connecting to MCP servers.

Your system does not need to become an MCP server itself (unless you decide to).

🧩 How MCP affects your architecture

Here is the delta — the changes needed to integrate MCP into your future agentic system.

1️⃣ Add an MCP Client Layer inside the Agent runtime

In your current plan, the Agent Orchestration Layer contains:

Supervisor Agent

Planner Agent

Tool Registry

Specialized Agents

Task Memory

Internal tools (Search, PDF generator, ingestion client…)

To support MCP, you add:

➜ MCP Client Module

This module:

Connects to external MCP servers

Authenticates to them

Discovers available tools

Registers those tools into your internal Tool Registry

Handles tool invocation requests from the LLM

Converts agent tool calls → MCP calls

Converts MCP results → agent-readable responses

The rest of your agent runtime remains unchanged.

2️⃣ Tools become “hybrid”: internal + MCP-external

Currently, your agent layer will have internal tools, e.g.:

SearchTool

DocumentFetchTool

MetadataFilterTool

PdfGenerationTool

With MCP support, your Tool Registry is extended:

ToolRegistry = {

internal_tools: [...],

mcp_tools: [...], <-- NEW

}

Your Supervisor Agent or Planner does not care where the tool comes from — the MCP client abstracts it.

---

3 RAG does not change

Your RAG stack stays exactly the same:

vector DB

embeddings

chunk metadata

re-ranking

context assembly

Agents may *use* the RAG engine as a tool, but MCP does not alter the retrieval architecture.

---

4 Your Q&A-only Chat API remains unchanged

MCP is only used by agents, not by simple Q&A.

So your existing:

POST /v1/chat/completions

remains untouched.

Agentic tasks go through:

POST /v1/agent/tasks

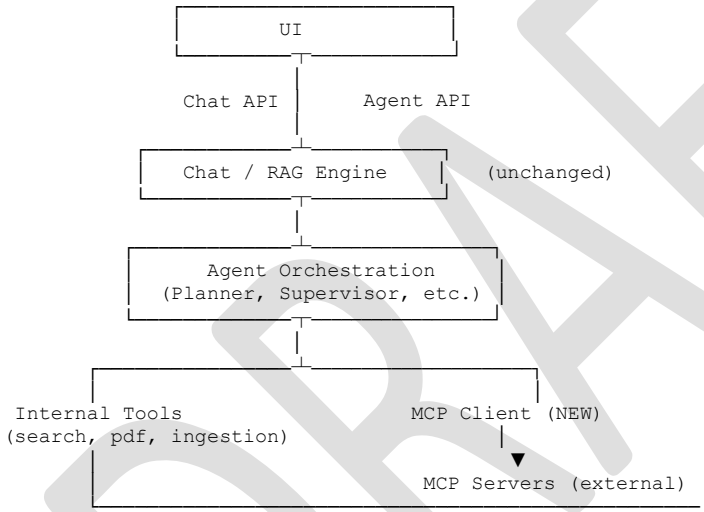
This separation is precisely what allows MCP integration without destabilizing the system.

5️⃣ Your Ingestion API also remains unchanged

MCP does not affect ingestion.

You might later expose ingestion as a tool via MCP if desired (e.g. "upload a document"), but it's optional.

📐 Updated architecture diagram (high-level)



This shows clearly:

You only *add* the MCP Client

No other subsystem is altered

🎯 **So what must you prepare now?**

Just **three small design decisions**:

✓ 1. Keep the Agent API separate from the Chat API

You already planned this.

This allows:

Chat API → stateless, synchronous

Agent API → multi-step, MCP-driven, asynchronous

Perfect for MCP.

✓ 2. Design your Tool Registry to be pluggable

Don't hardcode your tools.
Use a simple interface:

Python

Your MCP Client will implement multiple Tools through that interface.

✓ 3. Ensure your agent runtime supports structured tool calls

MCP uses structured tool invocation (e.g. JSON-schema based).
Most modern LLMs already support function-calling/tool-calling JSON formats, so this aligns perfectly.

Just make sure your agent supervisor can route:

internal tool calls

MCP tool calls

using the same interface.

🏁 Final answer

Adding MCP support requires minimal changes:

- ✔ **Add an MCP Client module inside the Agent layer**
- ✔ **Extend your Tool Registry to include MCP tools**
- ✔ **Keep Chat API and Agent API separate (you already do this)**
- ✔ **No changes required to:**

RAG engine

ingestion pipeline

vector DB

model layer

metadata schema

basic chat system

Your architecture is already MCP-ready.
You simply plug MCP servers into the new agent layer when needed.

If you want, I can also generate:

✅ A new **clean layered Mermaid diagram with MCP included**
✅ A **sequence diagram** showing an agent calling an MCP server
✅ A **migration plan**: from Q&A-only → Agentic → MCP-enabled agents
Just tell me!

18 Scalability considerations

Let's say that at some point in the future the system has to be deployed in a large business where many users use it simultaneously. How should the design take into account possible, future scalable deployments for such scenarios?

Copilot said:

Copilot

Yes — if you expect the system to serve many concurrent users in a large business, the design should be made scalable by separation of concerns, so that you can scale the API layer, model-serving layer, retrieval layer, and ingestion/background-processing layer independently instead of scaling one monolith. Kubernetes' Horizontal Pod Autoscaler (HPA) is designed precisely for this style of architecture, where different workloads can scale horizontally based on CPU, memory, or custom metrics.kubernetes+1

The good news is:

You do not need a completely different architecture later.
You need to design the current system with the right boundaries now, so each subsystem can be scaled separately in the future.kubernetes+2

---

1) Design principle: split the system into independently scalable services

For future enterprise-scale deployment, I would explicitly separate the system into these deployable units:

Frontend / API services

RAG orchestration service

Model-serving service(s)

Vector database / metadata database

Ingestion and background worker services

Object/file storage

This separation matters because user-facing request load, embedding throughput, LLM generation throughput, vector search throughput, and ingestion throughput scale very differently. Kubernetes HPA scales workloads such as Deployments or StatefulSets horizontally based on observed metrics, which is exactly what you want once these concerns are isolated.kubernetes+1

2) Keep the API layer stateless so it can scale horizontally

Your future Chat API and Ingestion API should be implemented as stateless services as much as possible. That means:

no in-memory user session state that is required for correctness,

no local-only task state,

no local-only document state,

no sticky dependency on one process instance.

FastAPI's deployment guidance explains that you can run multiple worker processes to handle more requests, and also notes that in container/Kubernetes deployments it is common to run a single Uvicorn process per container and let the orchestration layer handle replication.tiangolo+1

Practical implication

For future scale, design now so that:

session state lives in a shared store (DB/Redis), not inside a web process,

task status lives in a shared store,

uploaded files go to object storage or shared volume,

the API pods can be replicated freely.

That makes the API layer easy to scale with HPA later. Kubernetes HPA can then add or remove API pods based on CPU, memory, or custom metrics.kubernetes+1

---

3) Separate the model-serving layer from the application layer

The LLM/embedding/vision models should not live inside the same processes as the web app once you move to serious multi-user traffic.

Instead, think in terms of:

API service → validates requests, manages sessions, orchestrates RAG

model-serving service → serves chat model, embedding model, reranker, OCR/vision model

This is important because model-serving has very different scaling behavior and hardware needs.

Why this matters

vLLM's scaling documentation distinguishes:

single GPU when a model fits on one GPU,

tensor parallelism for multi-GPU on a single node,

tensor + pipeline parallelism for multi-node serving,

data parallel deployment where model weights are replicated across multiple instances/GPUs to process independent batches of requests.vllm+1

That means future scale should look like this:

small traffic → 1 model replica

higher traffic → more replicas via data parallelism

bigger models → multi-GPU / multi-node via tensor/pipeline parallelism

vLLM also notes that for online deployments, data-parallel engines benefit from intelligent load balancing that considers queued requests and KV cache state.vllm

Practical implication

Design your current system so that:

the chat backend calls models through a model gateway / model service endpoint

the UI never directly depends on a specific model process

swapping from local Ollama to remote vLLM later does not require redesign of the chat flow

That is one of the most important future-proofing decisions.

---

4) Plan for the vector database as a clustered service, not a local component

For a large business deployment, the vector store should be treated as a first-class distributed service.

Qdrant's production guidance is very clear:

a single node is best only for non-production workloads,

replicated 3+ node clusters are the recommended production standard,

replicated clusters provide better resilience and higher performance through load balancing and redundancy.qdrant+1

Qdrant's distributed mode also supports:

sharding for horizontal scale,

replication for fault tolerance,

and cluster coordination for multi-node operation.qdrant+1

Practical implication

Even if your PoC starts with a single-node vector DB, design the storage interfaces so that later you can move to:

a Qdrant cluster

with shard replicas

and with enough nodes to tolerate failures.

If you anticipate enterprise traffic, the safest future target is a 3-node replicated cluster rather than a single-node vector DB.qdrant+1

---

5) Move heavy work to asynchronous workers early

This is one of the most important changes for future scalability.

Anything that is expensive or slow should not run synchronously inside the request/response path unless absolutely necessary. That includes:

document ingestion,

OCR / image enrichment,

reindexing,

PDF/report generation,

large summary builds,

future agentic workflows.

Redis' own job-queue tutorial recommends using Redis Streams + consumer groups for background workers, with retries and dead-letter handling.redis

Practical implication

For future enterprise use, I would recommend this pattern:

API receives request

enqueue job

return quickly with task ID

worker processes do the heavy work

task result is stored and later fetched

This applies both to:

ingestion pipeline jobs

and future agent/report-generation jobs

A proper queue/worker architecture protects the user-facing APIs from being blocked by long-running tasks and lets you scale worker pools independently from chat traffic. Redis Streams with consumer groups are specifically designed for this kind of background-processing workflow.redis

---

6) Use Kubernetes autoscaling as the future control plane

If you eventually deploy for many simultaneous users, Kubernetes is the most natural control plane for this kind of system.

Kubernetes HPA:

automatically scales workloads horizontally,

can use CPU, memory, or custom metrics,

periodically adjusts replica counts for Deployments/StatefulSets.kubernetes+1

The Kubernetes autoscaler project also includes:

Cluster Autoscaler for adjusting node count,

Vertical Pod Autoscaler for adjusting CPU/memory requests.github

Practical implication

Your future enterprise deployment should be prepared to scale at three levels:

A. API replicas

Scale Chat API / Ingestion API pods with HPA.kubernetes

B. Model-serving replicas

Scale model servers separately from API replicas, using GPU-aware placement and possibly custom metrics. HPA supports custom metrics in addition to CPU/memory.kubernetes+1

C. Cluster nodes

Use cluster autoscaling so that more nodes are added when more API pods / worker pods / model pods need to be scheduled. Kubernetes autoscaler components are built for exactly this style of dynamic capacity management.github

---

7) Think in terms of concurrency budgets, not just “number of users”

For large-user scenarios, your bottleneck will usually not be “user count” by itself, but a combination of:

concurrent requests,

average prompt length,

average response length,

retrieval fan-out,

vector DB latency,

model KV cache pressure,

ingestion jobs running at the same time,

file/OCR workloads.

vLLM explicitly exposes useful capacity indicators such as:

GPU KV cache size

maximum concurrency estimate for a given tokens-per-request setting. It recommends adding more GPUs or nodes when those figures are too low for your throughput requirements.vllm

Practical implication

In the future, capacity planning should be done using:

expected concurrent chat sessions,

average request/response token size,

peak QPS,

ingestion throughput,

and vector search latency targets.

This is why I would keep the model-serving layer and retrieval layer independent from the API layer from the beginning.

---

8) Build for resilience, not just throughput

In a large business, downtime and inconsistent behavior matter as much as raw speed.

Qdrant's distributed deployment guidance highlights that:

3+ node replicated clusters can continue operating even when one node is down,

and can recover from permanent loss of one node without relying only on backups, although backups are still recommended.qdrant

FastAPI deployment guidance also emphasizes replication and restart strategies when running production servers with multiple processes.tiangolo+1

Practical implication

Future scalable deployment should include:

replicated API services,

replicated worker services,

replicated vector DB nodes,

object storage not tied to a single VM,

and model-serving replicas or failover strategy.

In other words, future scale should be designed for high availability as well as high concurrency.

---

9) Add observability and custom metrics from the start

At enterprise scale, you will need to know:

where latency comes from,

when the vector DB is the bottleneck,

when model-serving is saturated,

when ingestion backlog grows,

when access-control filters impact retrieval cost.

Qdrant's operations docs explicitly include Monitoring & Telemetry and Prometheus integration as production concerns.
Kubernetes HPA can also scale on custom metrics, not just CPU/memory.qdrantkubernetes+1

Practical implication

Even before you need full production monitoring, define the internal metrics you will eventually want:

API latency

retrieval latency

reranker latency

model tokens/sec

vector DB query time

queue depth

ingestion job duration

task failure rate

That way, when you later add observability, the architecture already has the right boundaries.

---

10) A practical “design now for scale later” checklist

Here is the design checklist I would recommend for your current system so it can grow into a large-business deployment later:

Keep now

One Chat API

One Ingestion API

One RAG core

One vector DB

One model abstraction layer

Make scalable later by design

Stateless APIs with shared session/task state

Dedicated model-serving layer instead of in-process models

Dedicated worker queue for heavy jobs

Cluster-ready vector DB interface

Object storage abstraction instead of local-only file assumptions

Containerized services so they can move to Kubernetes

Metrics and tracing hooks even if lightweight at first

This is the right compromise: you avoid over-engineering the PoC, but you also avoid dead-end design decisions.

My recommended future target architecture

If this becomes a serious internal platform, I would aim for something like:

Frontend / Streamlit or Web UI

Chat API service (replicated)

Ingestion API service (replicated)

Background worker service (replicated)

Model gateway

vLLM serving layer for chat / embedding / rerank / vision, scaled independently with data parallelism and, if needed, tensor/pipeline parallelism.vllm+1

Qdrant cluster with replication and sharding.qdrant+1

Redis Streams or equivalent queue for ingestion/report jobs.redis

Kubernetes + HPA + cluster autoscaler as the deployment control plane.kubernetes+1

Bottom line

Yes — if you may later deploy in a large business with many simultaneous users, the design should take scalability into account now, but mostly by enforcing clean service boundaries, stateless APIs, background job execution, separate model serving, and cluster-ready storage. Kubernetes HPA, vLLM's distributed/data-parallel serving model, Redis-style job queues, and a replicated Qdrant cluster are all aligned with that direction.kubernetes+4

If you want, I can next generate one of these:

a layered Mermaid diagram for the future scalable enterprise deployment,

a migration roadmap from local PoC → remote server → Kubernetes cluster, or

a technology-specific reference architecture (e.g. FastAPI + vLLM + Qdrant + Redis + Kubernetes).

19 Duplication avoiding for unstructured data

How to avoid data duplication in the knowledge base if I use also unstructured data such as audio recordings to build it?

Copilot said:

Copilot

You *can* add audio (or any unstructured modality: images, video, OCR results, speech-to-text transcripts) to your knowledge base without duplicating data—but only if you design a canonical document model and a linking strategy that makes all modalities part of a *single* logical document.

Below is a clear, practical design used in scalable enterprise RAG systems to eliminate duplication.

✅ The core idea

Never treat each modality as a separate document.
Instead, treat them as different "representations" of one canonical knowledge object.

That means:

One canonical document ID

Multiple representations (text, audio, transcript, images, metadata)

Chunk-level embeddings refer back to the same original source

This prevents the same content from being indexed as if it were multiple different documents.

🧱 1. Introduce a Canonical Document Model

Here is the key structure:

JSON

All chunks produced from any modality must reference the same doc_id.

This eliminates duplication by design.

🎧 2. Avoid "double indexing" by marking derived text as *representations*

The biggest duplication problem with audio files is:

You embed the original transcript → embeddings 1

You also separately ingest the original audio metadata → embeddings 2

To avoid this:

✔ Only embed the *most meaningful representation*

For audio, that is usually the transcript (possibly improved with LLM summarization or normalization).

✔ Do NOT embed multiple representations blindly

For example:

raw ASR output

cleaned transcript

LLM-paraphrased transcript
…should not all be indexed as independent chunks.

Instead, keep only *one* canonical source representation, e.g.:

canonical_text = LLM.cleaned(transcript)

🎯 3. Use chunk metadata to unify all content from one audio file

Each chunk created from the transcript should carry:

JSON

This gives two benefits:

Retrieval can de-duplicate chunks by doc_id

The user gets time-stamped citations pointing to the original audio

---

🔎 4. Use metadata filtering to prevent duplication during retrieval

When a query hits multiple chunks from various modalities of the *same* document, you can:

collapse results by doc_id

or dedupe at ranking stage

This is trivial if your vector DB stores doc_id in chunk metadata:

Example de-duplication logic (Reranker Stage)

group chunks by doc_id

→ choose best N chunks per document

→ rerank

→ answer

This ensures that a single audio file never overwhelms the vector search results.

🔁 5. Prevent repeated ingestion of the same content

When you support ingestion of different formats (PDF, audio, images, HTML), duplication risk increases.

Fix this by:

A. Give each canonical document a stable doc_id

Possible strategies:

stable filename

user-provided document title

content hashing (hash of audio file or transcript)

B. Maintain a Document Registry

A small metadata DB table like:

doc_id | hash | source_type | last_modified | representations

Before processing:

compute hash

check if this file has been ingested before

skip or update instead of duplicating

This prevents uploading the same training recording twice.

📦 6. Use an object storage layout that mirrors logical documents

Example:

/knowledge-base/

/battery_safety_2025/

audio.wav

transcript.txt

metadata.json

enriched_summary.md

This structure makes the canonical relationship clear and helps avoid ingesting each file type as if it were independent.

🧠 7. Enable cross-modality merging during ingestion

When you ingest audio, PDF, images, etc., you should decide:

Does this extend an existing document?

Or does it create a new canonical document?

Example:
An audio training session referencing a product may belong to the same canonical object as the product's PDF manual.

You can do this manually now, and automate later using:

filename patterns

embeddings similarity

explicit user choice

⚙️ 8. If you use "Unstructured", "Docling", or "Whisper" — unify the outputs

Unstructured.io, Docling, and Whisper-style ASR pipelines produce different modalities. The trick is to:

feed all outputs into your canonical document builder

then chunk and embed from the canonical text representation only

attach the original modality objects as metadata

This avoids the common mistake of embedding *every representation* separately.

🏁 Summary: The rules that eliminate duplication

To keep your knowledge base clean:

Rule 1 – Canonical Document ID

Every piece of content (audio, transcript, PDF, image) must resolve to a single document identity.

Rule 2 – One text representation indexed

Choose one canonical text form and embed only this.

Rule 3 – Multi-modality recorded as metadata

Different data types enrich the document but do not create new documents.

Rule 4 – De-duplicate at ingestion via hashing + registry

No two files representing the same content should create new embeddings.

Rule 5 – Deduplicate at retrieval via doc_id grouping

Collapse results to avoid over-ranking one multi-modal document.

Want a visual architecture diagram?

If you want, I can generate a Mermaid diagram showing:

canonical document model

multi-modal ingestion

deduplication flow

retrieval dedupe

user-facing citation flow

Just tell me and I'll produce it!